



Computer Vision for Autonomous Vehicles

Loop Closure and Place Recognition

Instructor: Golnaz Habibi, PhD

University of Oklahoma

Fall 2025

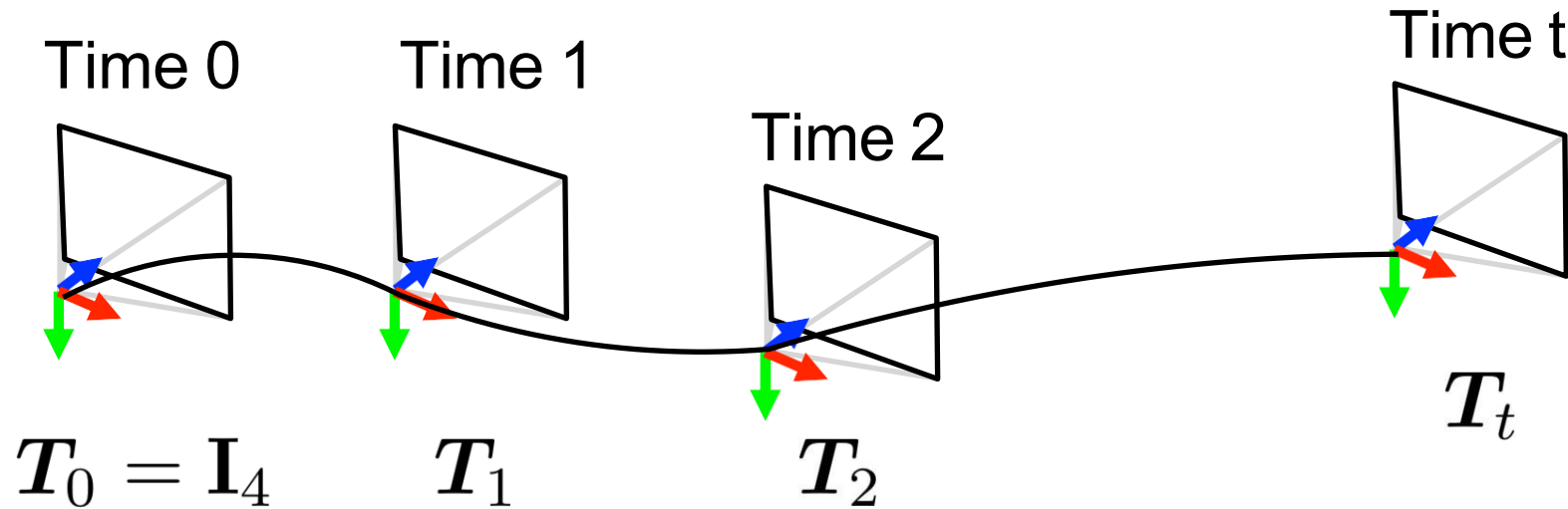


Announcement

- Lab 6 will be posted today, **due November 11**
- Lab 6 will be the last lab!
- Lab 7 (control) is considered as optional, if you do it, the lowest grade will be dropped.
- Final exam is take home and will be the combination of questions and analyzing and comparing different SLAM algorithms
- Today and next Lecture:

Visual Odometry

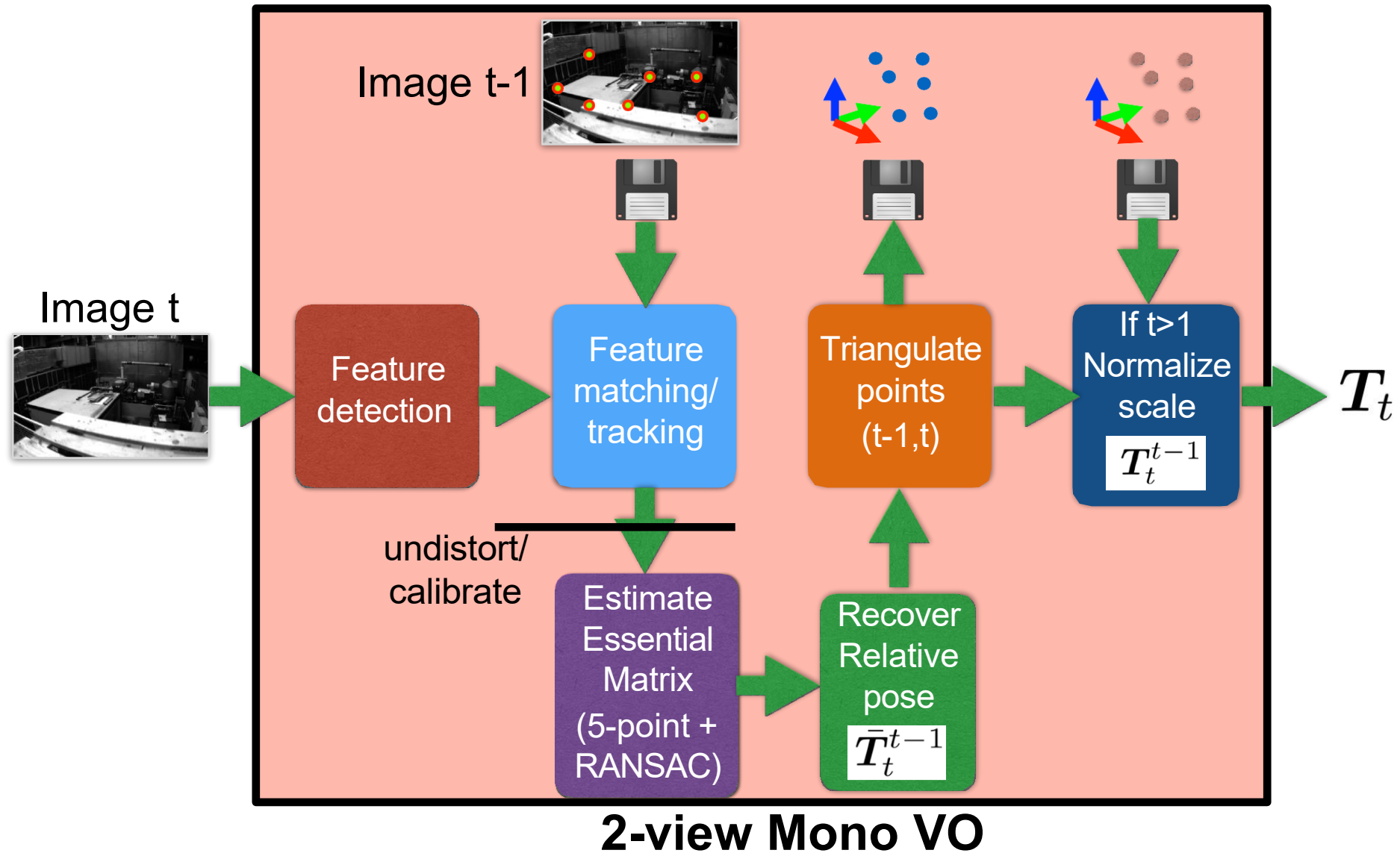
odometry: incremental motion estimation



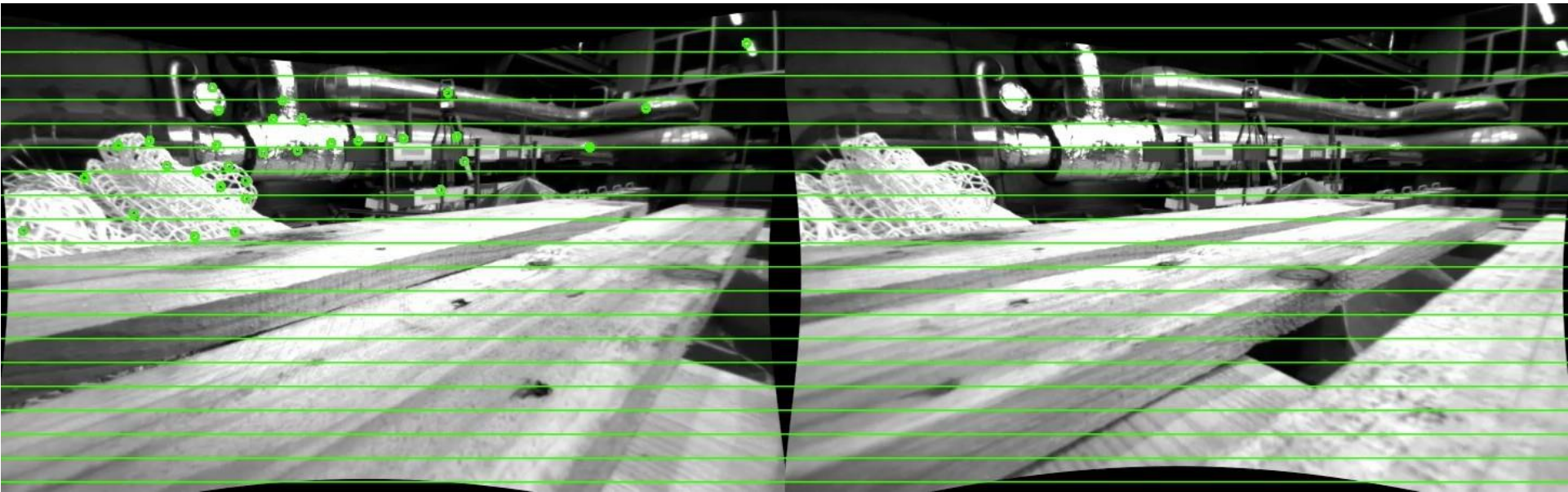
Visual odometry (VO): motion estimation estimation based on cameras (monocular, stereo, RGB-D, ...)

others: wheel odometry, inertial, visual-inertial

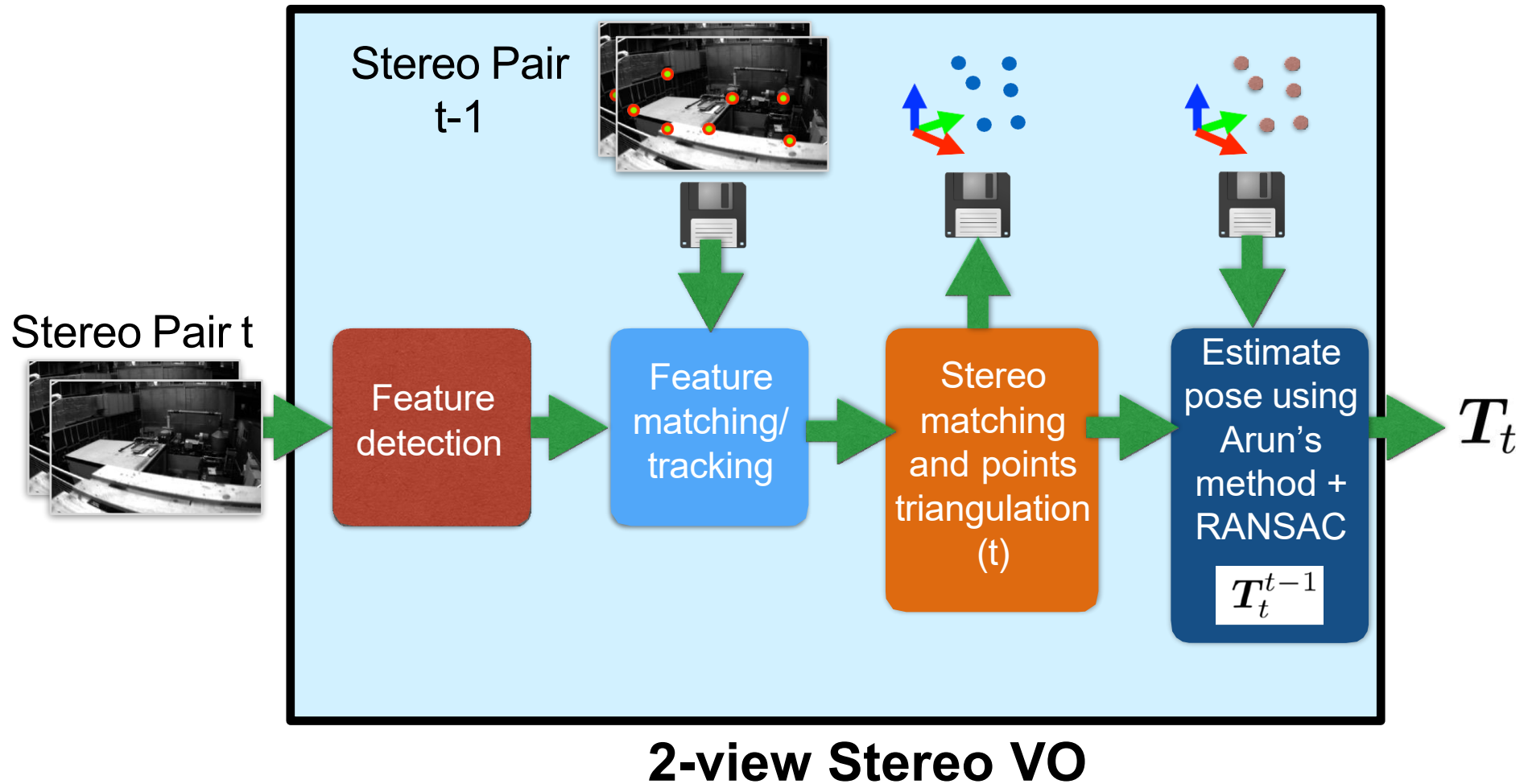
Monocular VO with 2D-2D Correspondences



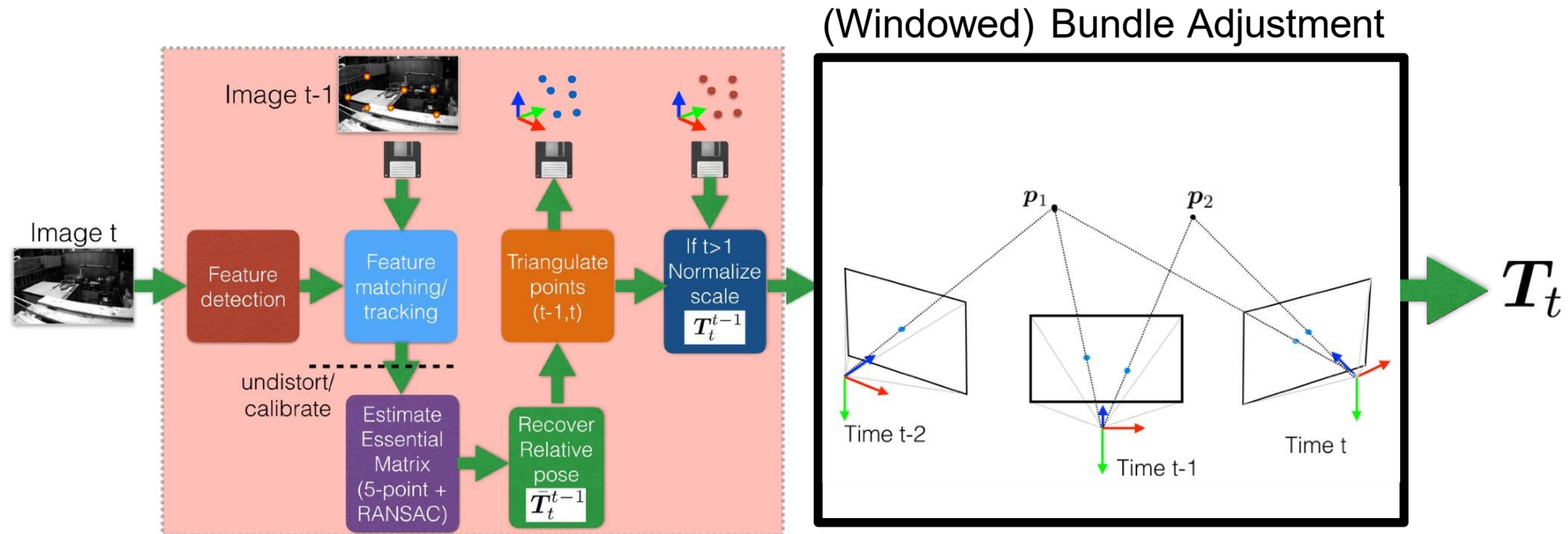
Stereo Matching



Stereo VO with **3D-3D** Correspondences



Refinement: Bundle Adjustment



Windowed Bundle Adjustment: optimization of the most recent camera poses and points via non-linear least squares

$$\min_{\substack{T_i, i=1, \dots, N_C \\ p_k, k=1, \dots, N}} \sum_{k=1}^N \sum_{i \in C_k} \|\mathbf{x}_{k,i} - \pi(\mathbf{T}_i, \mathbf{p}_k)\|^2$$

Can be applied to all the pipelines discussed today

Challenges for VO (1/3): Illumination and Features



Feature detection,
tracking,
matching ...



Challenges for VO (2/3): Dynamic Scenes

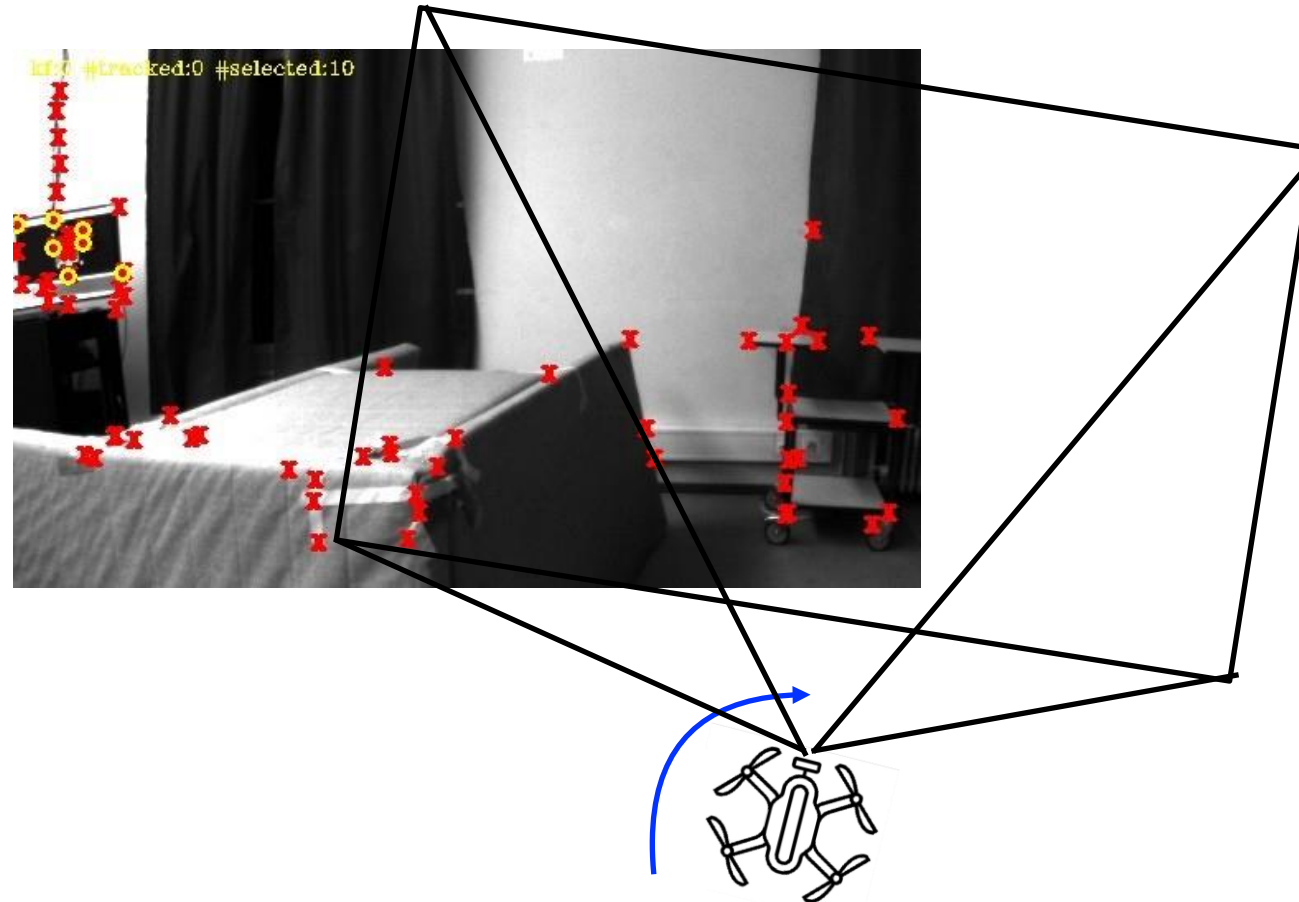
- Dynamic, crowded scenes present a real challenge
- Can't rely on RANSAC to always recover the correct inliers
- Example: Large van “steals” inlier set in passing



Inliers Outliers

Challenges for VO (3/3): Fast Motion

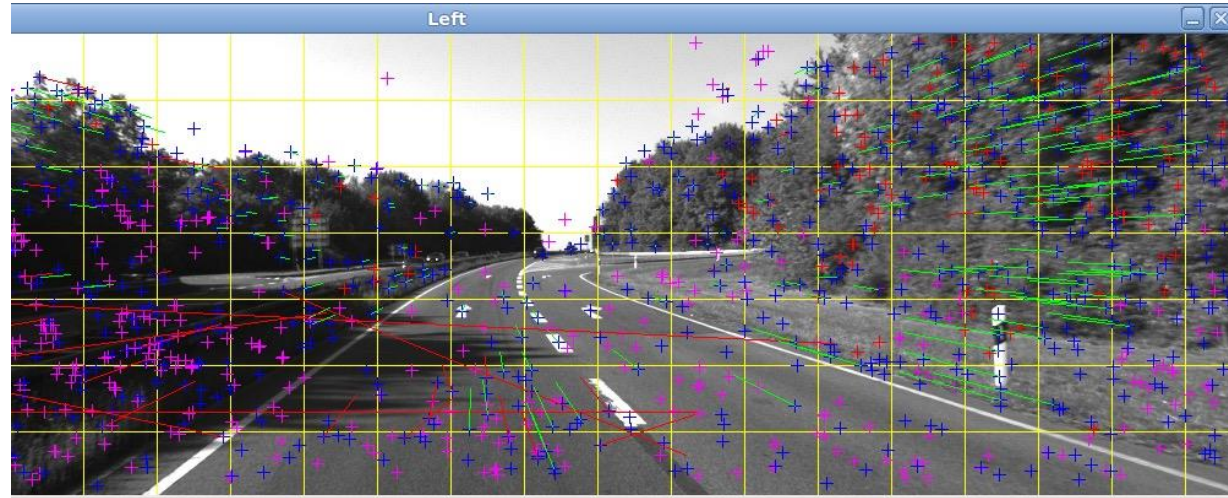
Need good overlap between consecutive images



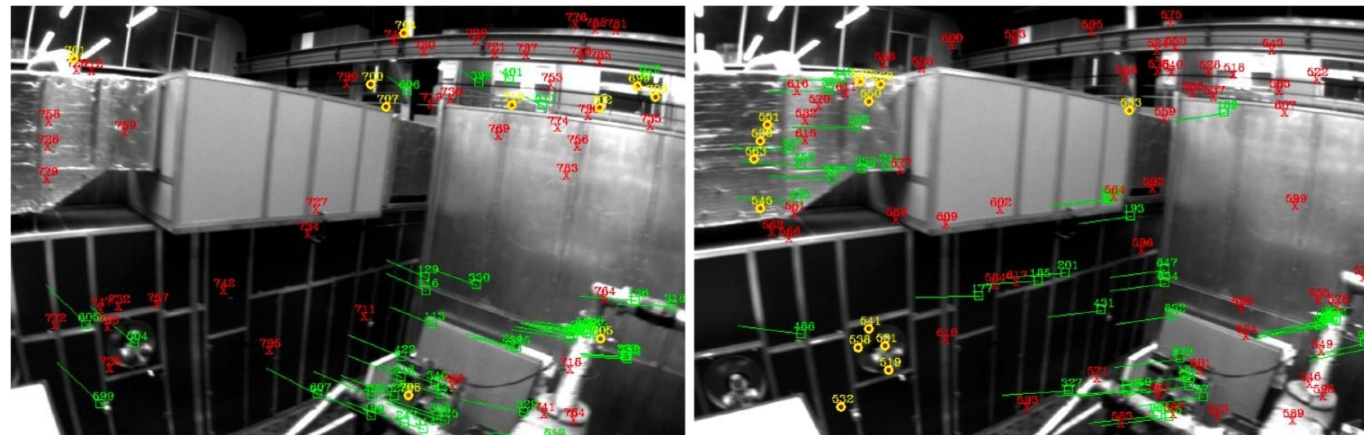
Robot speed, camera framerate, ...

VO Tricks (1/2): Feature Distribution

Feature Binning:



Attention & Anticipation:



select features depending on motion of the robot

VO Tricks (2/2): Domain Knowledge and Keyframes

- Stereo VO Example: Cross-traffic while waiting to turn left at light

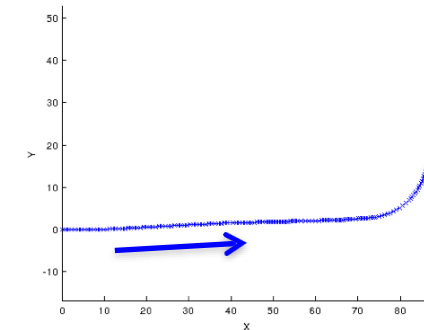
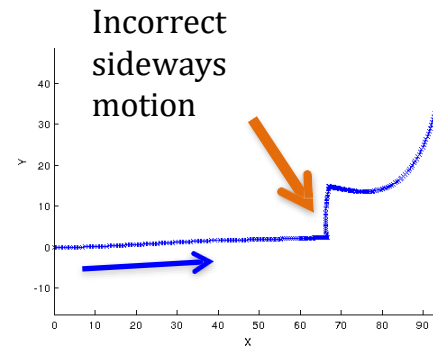


Only accept incremental pose if:

- Translation > 0.5m
- Dominant direction is forward



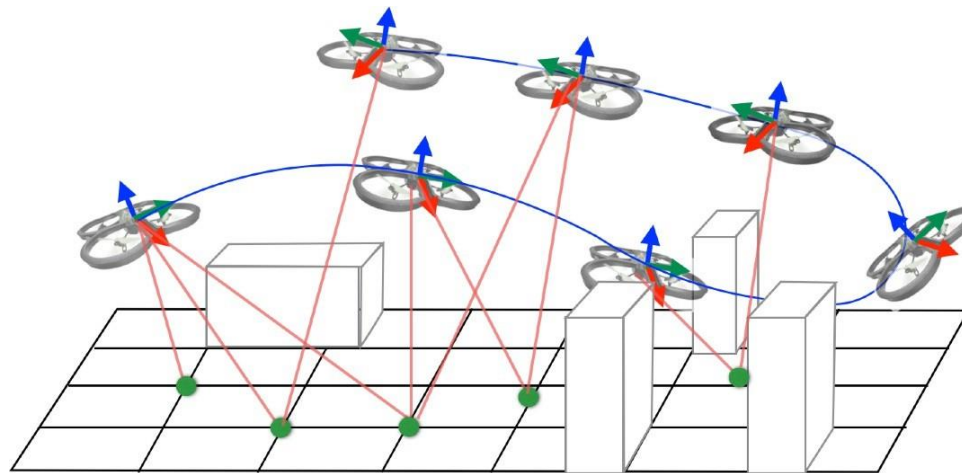
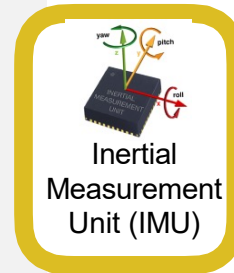
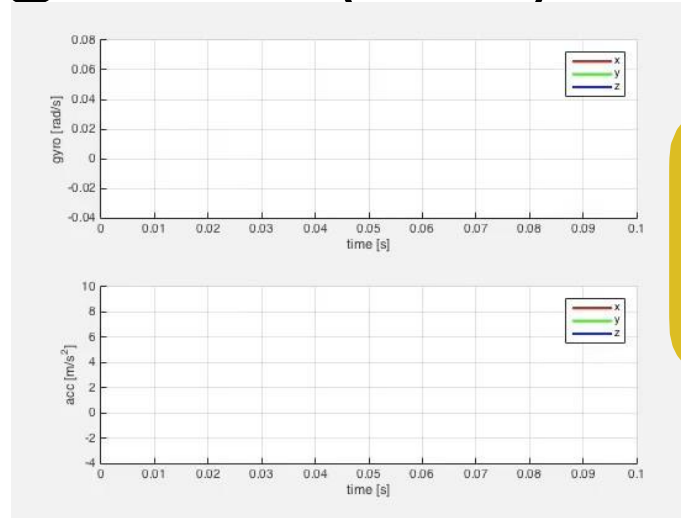
Without keyframing



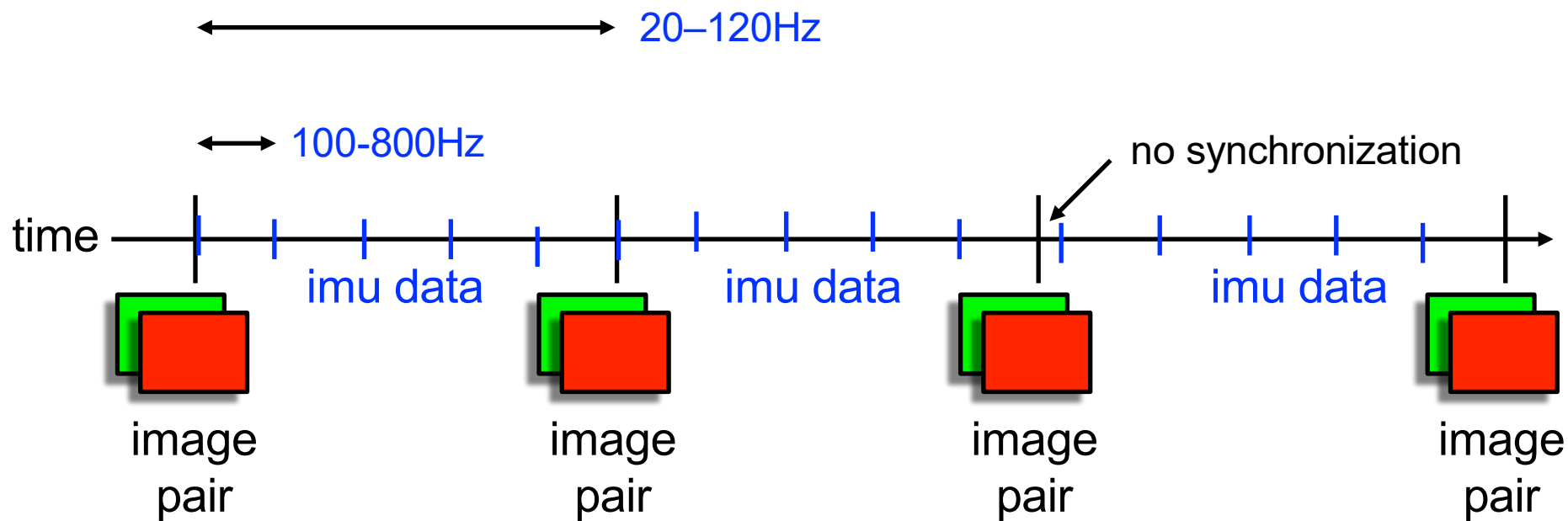
With keyframing

[courtesy of Frank Dellaert]

Visual-Inertial Navigation (VIN)



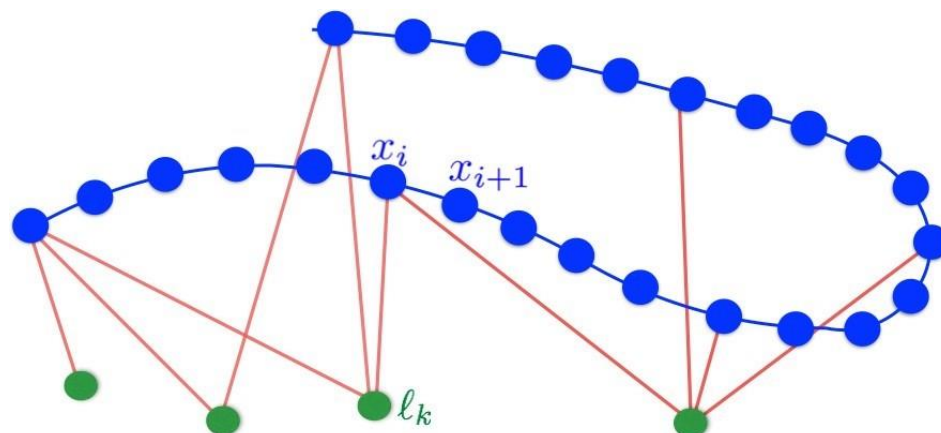
Visual-Inertial Odometry



Challenges:

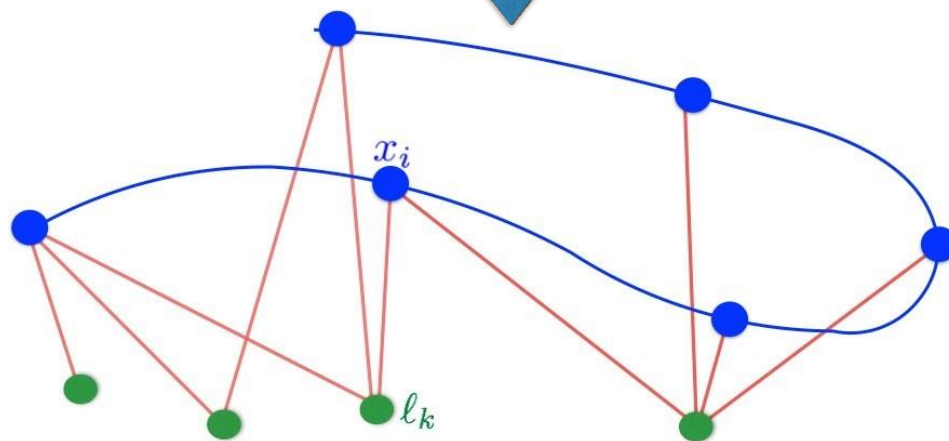
- IMU measurements arrive at high-rate ($\sim 200\text{Hz}$) ➡ IMU preintegration
- camera observes hundreds of landmarks per frame ➡ structureless vision factors
- need to solve optimization problem quickly

Pre-integration



After 10 seconds, original problem has $\sim 10^4$ states

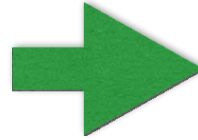
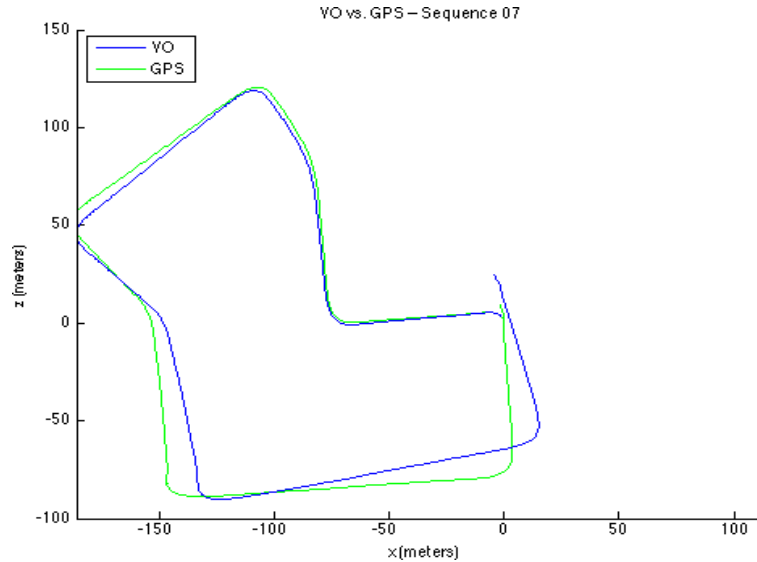
Preintegration



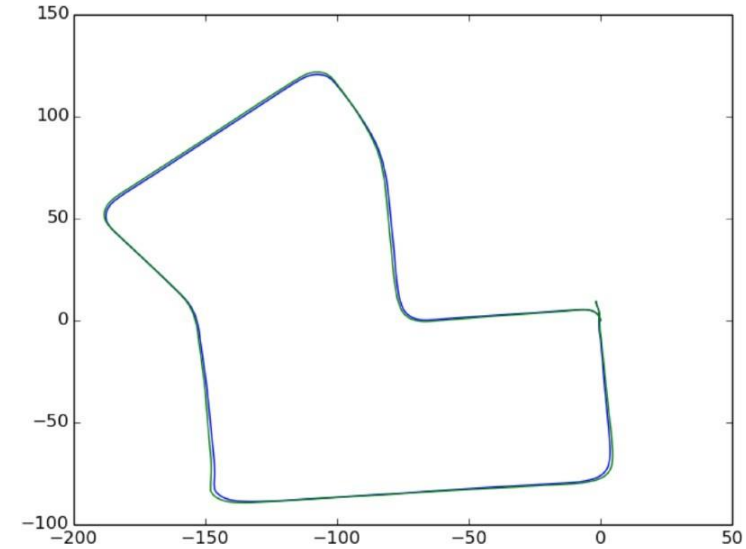
After 10 seconds, preintegrated problem has $\sim 10^2$ states

Removing Drift via Loop Closure

Visual(-inertial) odometry



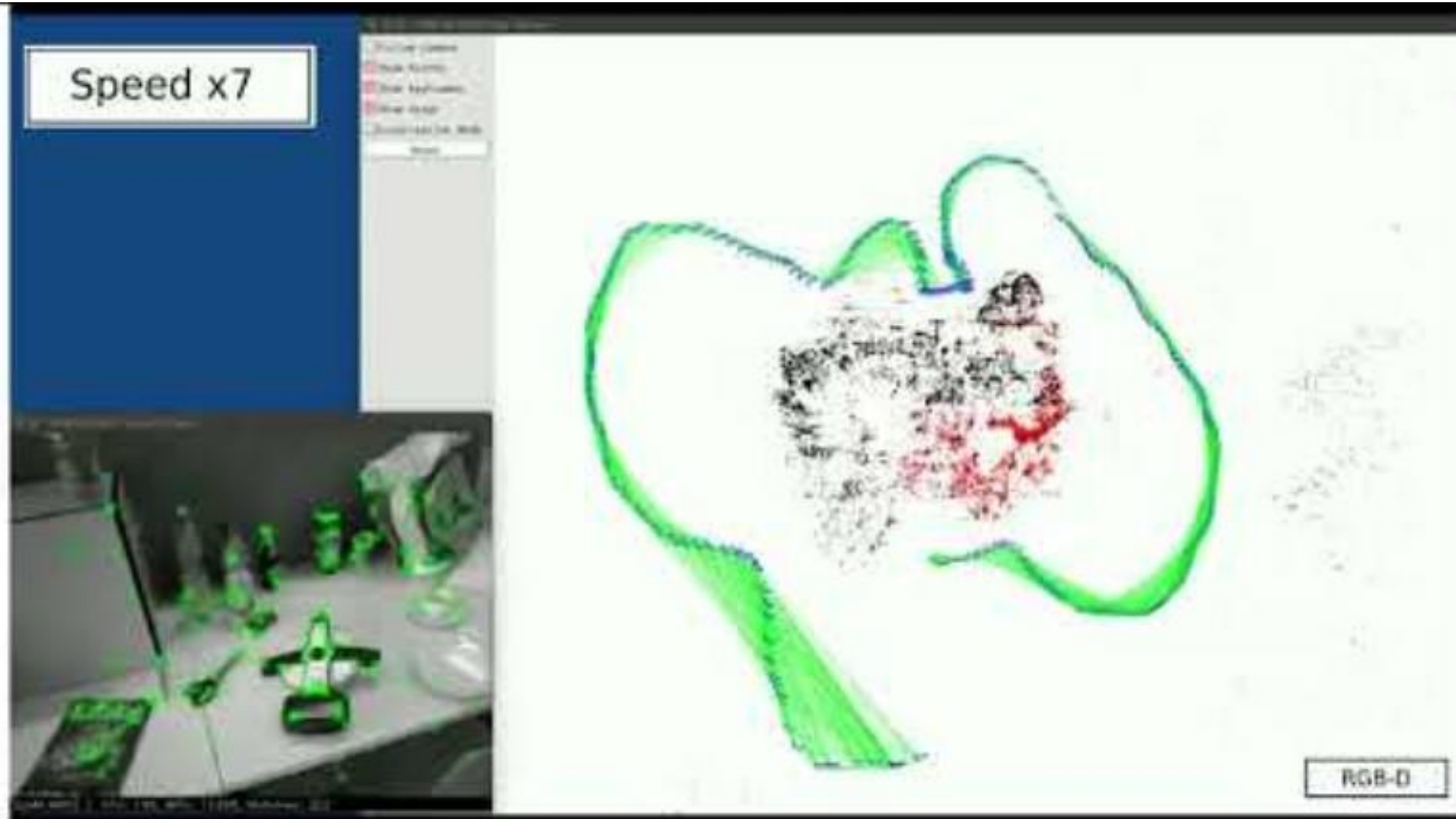
SLAM



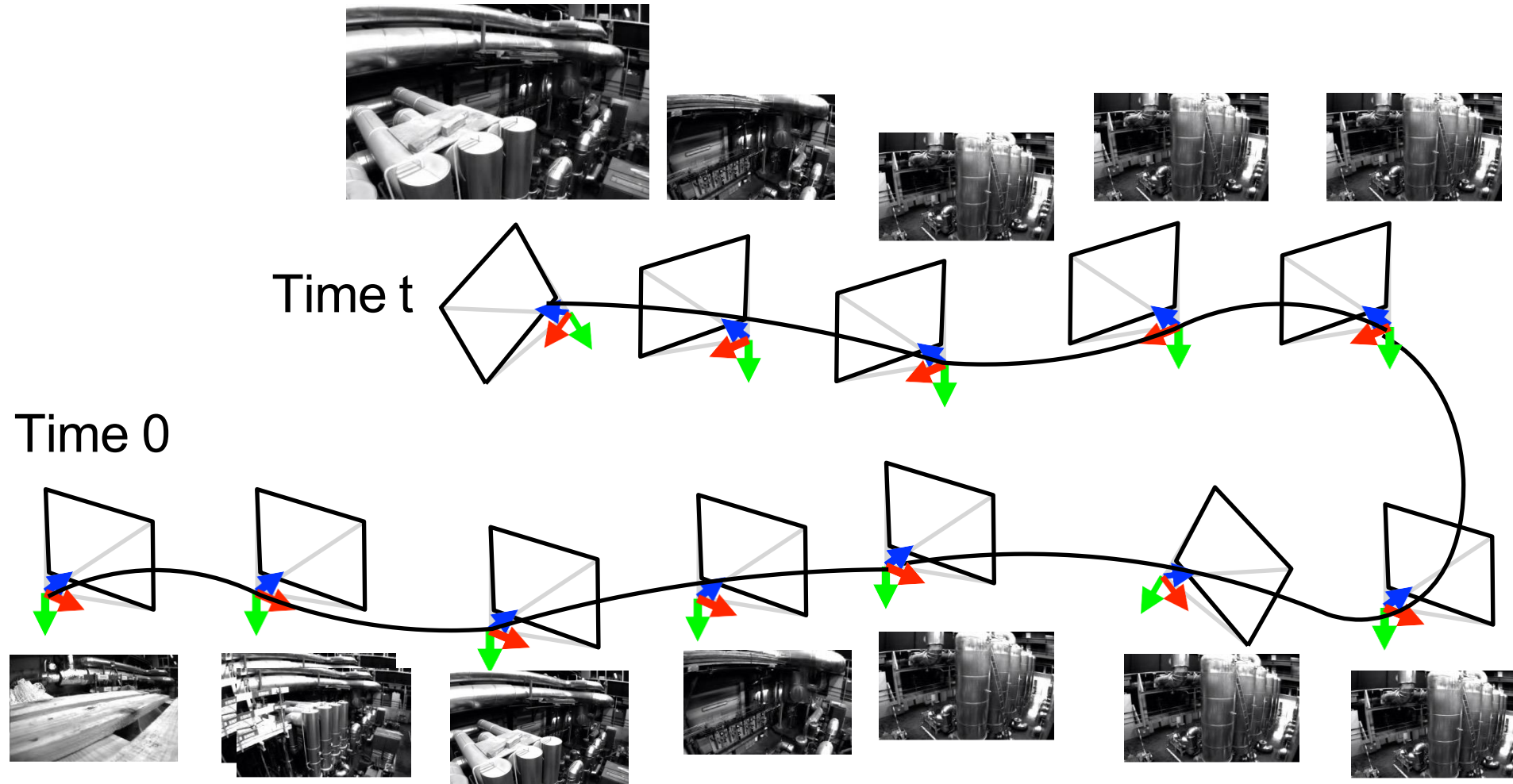
SLAM requires:

- place recognition (loop closure detection)
- Re-detecting landmarks (e.g., objects)

Need for loop closure



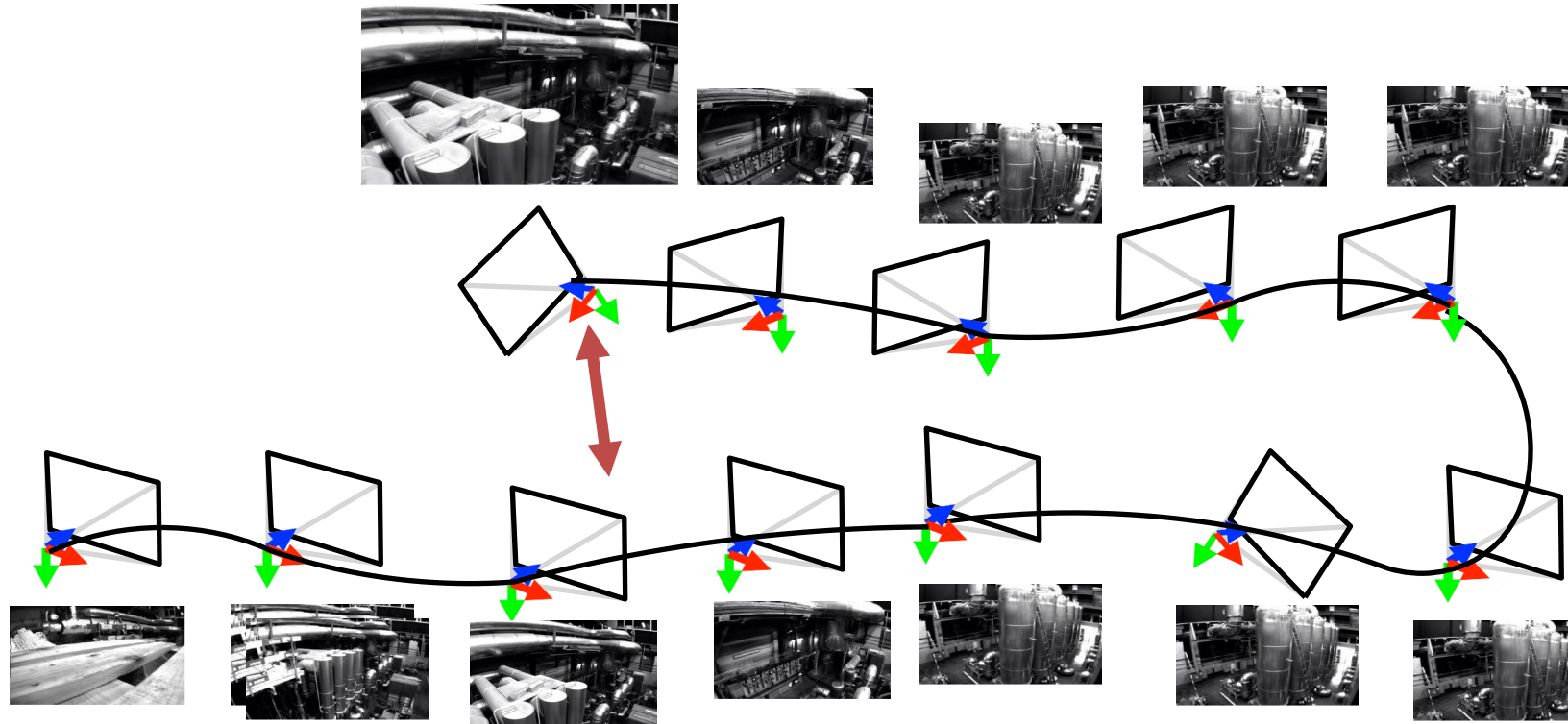
Place Recognition & Image Retrieval



Does the image at time “t” picture a place seen in previous images?

Place Recognition: formal definition

In place recognition, the goal is to determine where a current image (query) was taken by comparing it to a database of known places (reference images or maps).



Brute Force Place Recognition Steps

- Feature Extraction**

- Each image (query and database) is represented by descriptors (global or local).

 - Local*: SIFT, ORB, SURF

 - Global*: CNN features, NetVLAD, etc.

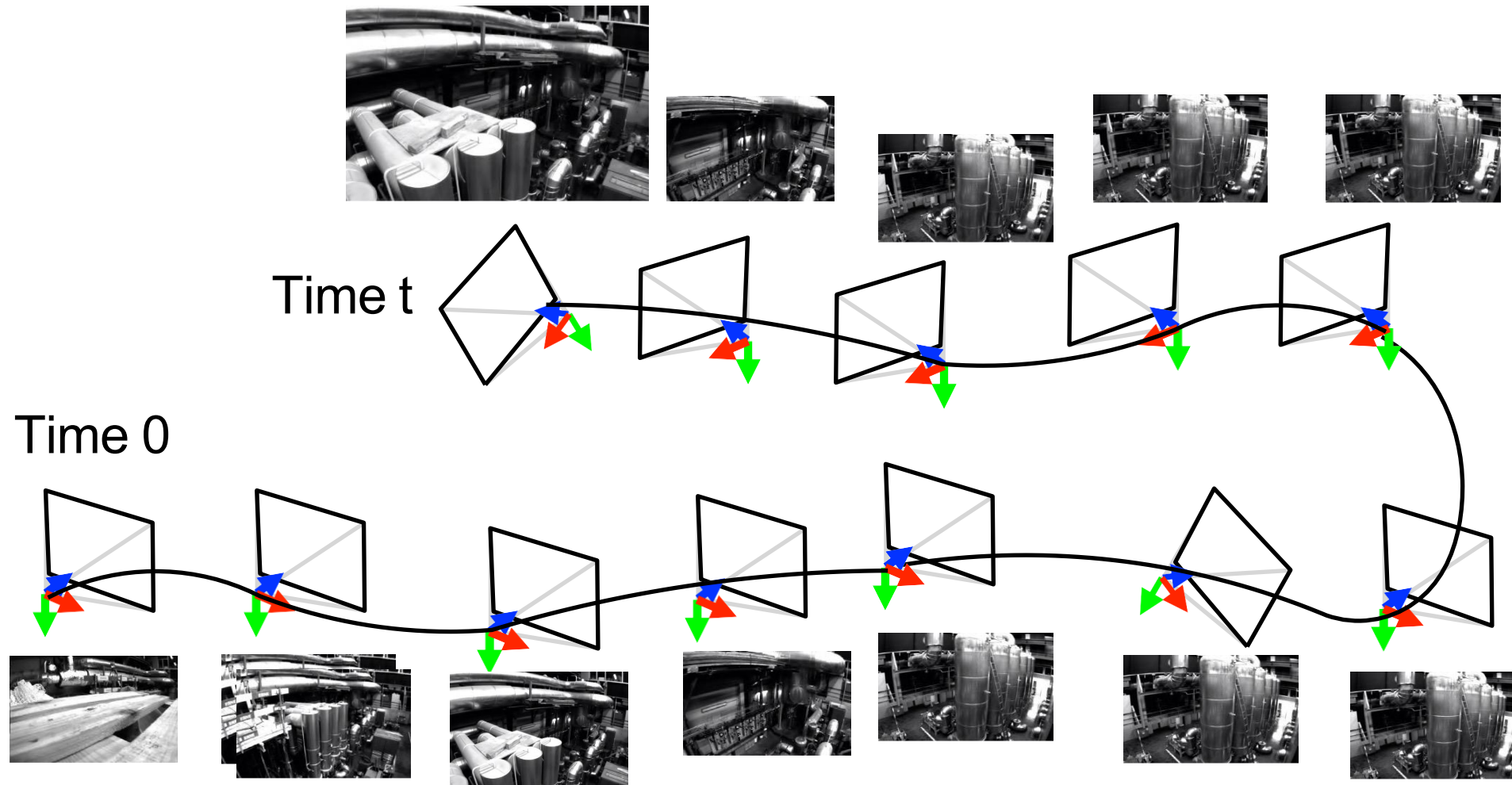
- Pairwise Matching**

- For a query image, compute the distance (e.g., Euclidean, cosine) between its descriptor(s) and **every database image descriptor**.

- Selection**

- The database image (or set of images) with the smallest distance is selected as the best match — i.e., the recognized place.

A brute force approach



Directly compare the query image to every image in the database using their descriptors, without any optimization or indexing.

Place Recognition: Challenges

- **Appearance changes:**

- Illumination
- Weather conditions
- Dynamic objects (people, cars,...)
- Viewpoint changes



- **Perceptual aliasing:**
two different places
may look similar
(building, roads, ...)

S. Lowry et al., "Visual Place Recognition: A Survey," in IEEE Transactions on Robotics, vol. 32, no. 1, pp. 1-19, Feb. 2016, doi: 10.1109/TRO.2015.2496823. © IEEE. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>



Bag of Words and Place Recognition



Guess the Speaker - Speech #1

[illegible]

Bag of Words (Natural Language Processing)

Representation:

- ▶ Build a **vocabulary**
- ▶ Represent documents as distributions (histograms) over the vocabulary

BoW : document $\mapsto$ histogram(document|vocabulary)

Document Retrieval:

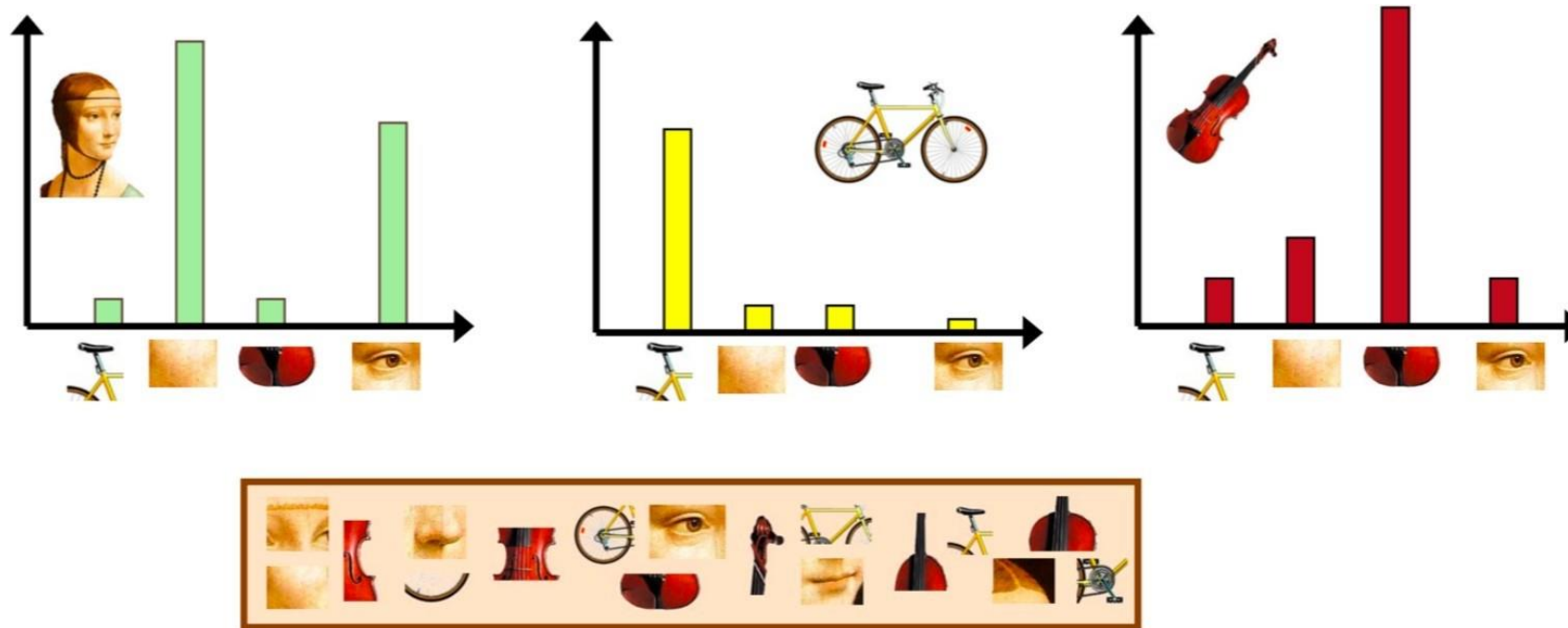
- ▶ Store the BoW histogram for every document in a DB
- ▶ Represent the **query** document as a histogram
- ▶ Compare the query histogram with histograms of documents in DB
- ▶ Return the best (or best n) matches
- ▶ Verify potential matches

Bag of Visual Words: Representation



Credit: Fei-Fei Li

Representation



Credit: Fei-Fei Li

Bag of Visual Words

Representation:

- ▶ Build a **visual vocabulary**
- ▶ Represent **images** as distributions (histograms) over the vocabulary

BoVW : **image** $\mapsto$ histogram(**image**|vocabulary)

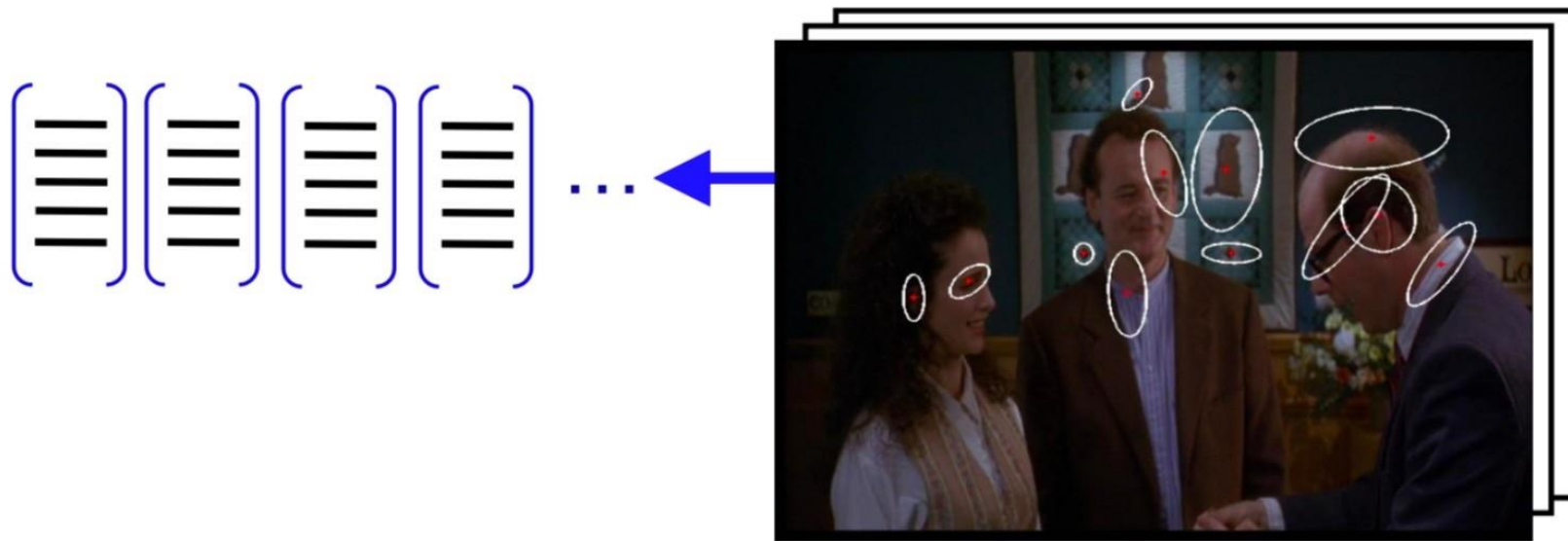
Image Retrieval:

- ▶ Store the BoW histogram for every **image** in a DB
- ▶ Represent the **query image** as a histogram
- ▶ Compare the query histogram with histograms of **images** in DB
- ▶ Return the best (or best n) matches
- ▶ Verify potential matches using **geometric/spatial** verification (**RANSAC**)

Build the Vocabulary

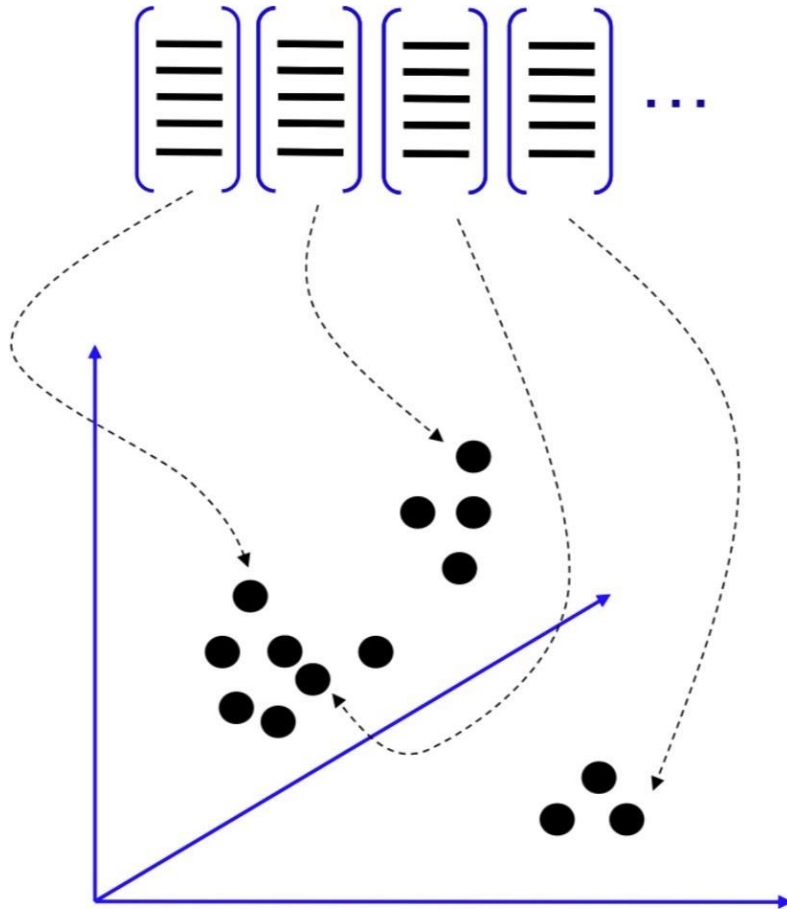
- ① Pick a set of images
- ② Extract keypoints and their descriptors from every image
 - ▶ Need to be fast, invariant to viewpoint variations, etc.
- ③ Cluster the descriptors into k clusters (using, e.g., k -means)
- ④ Pick the k cluster centers as your vocabulary

1- Extract Keypoints and Descriptors

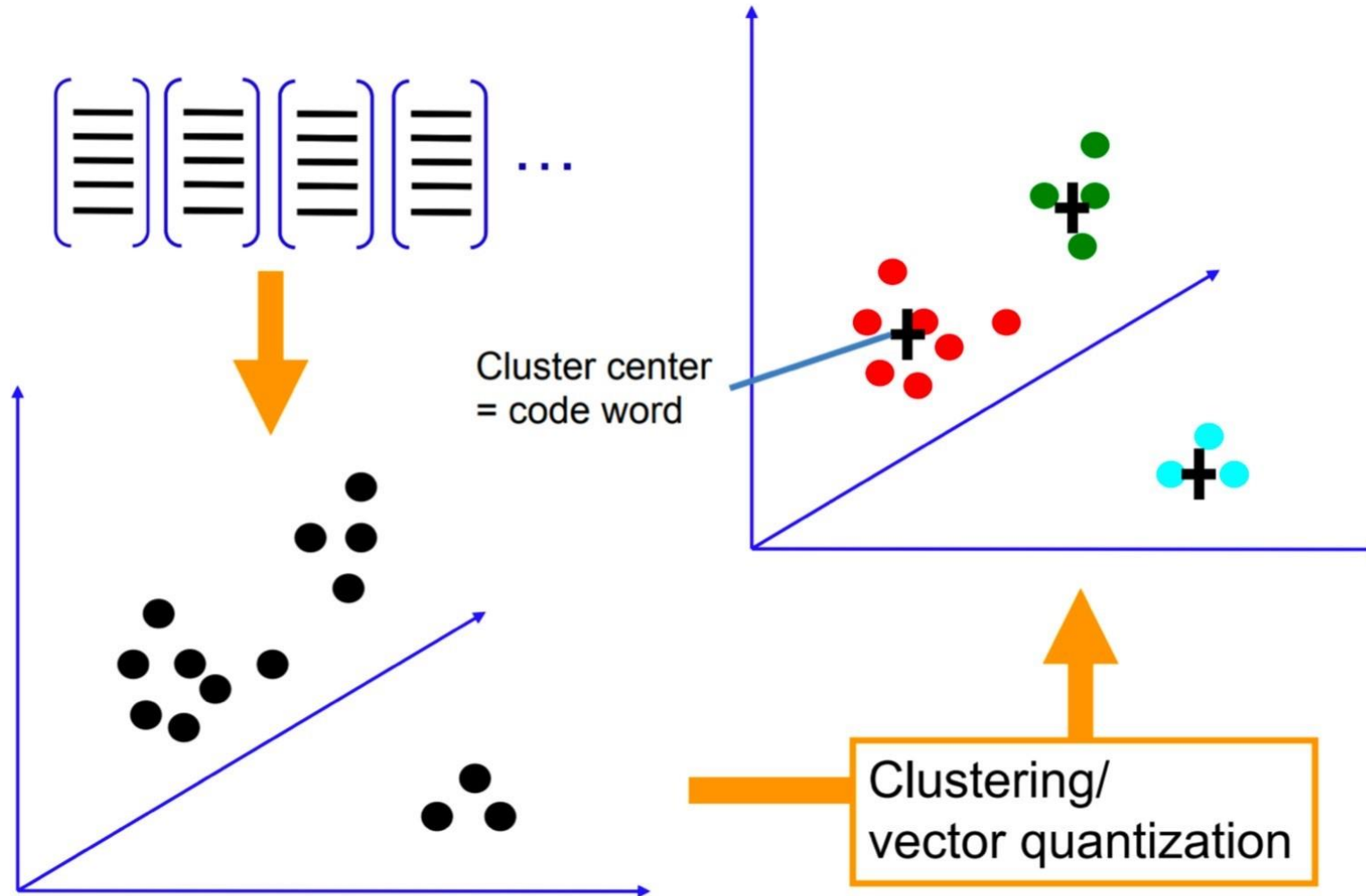


Credit: *Fei-Fei Li*

2- Descriptor Space



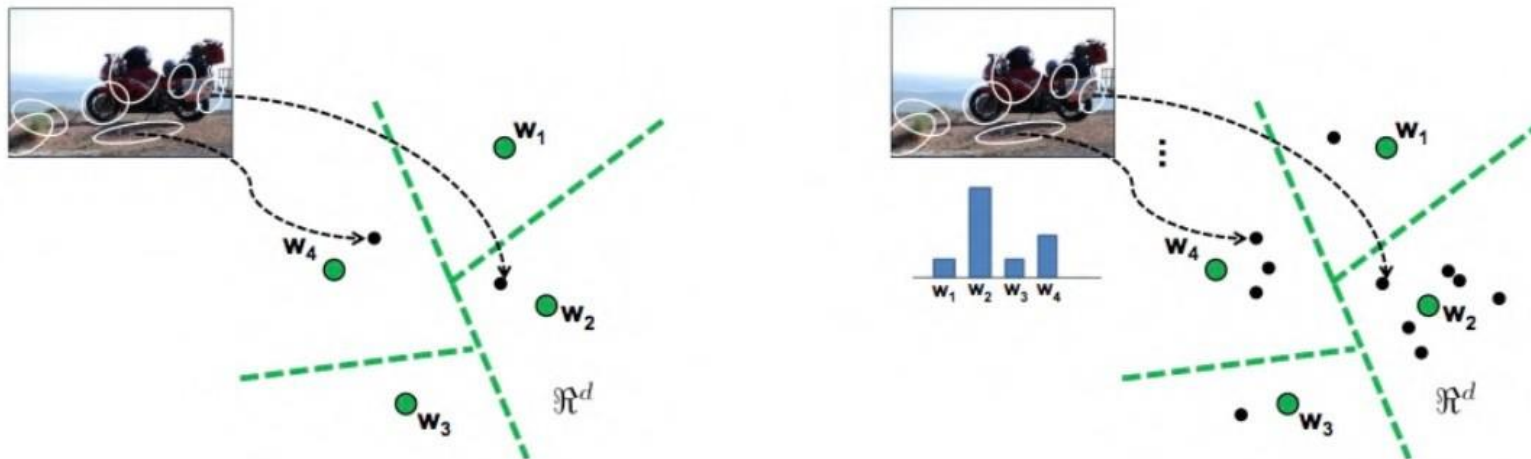
3- Cluster the the Descriptors to Build the Vocabulary



Local descriptors: Bag of Words

Based on text retrieval and summarization methods

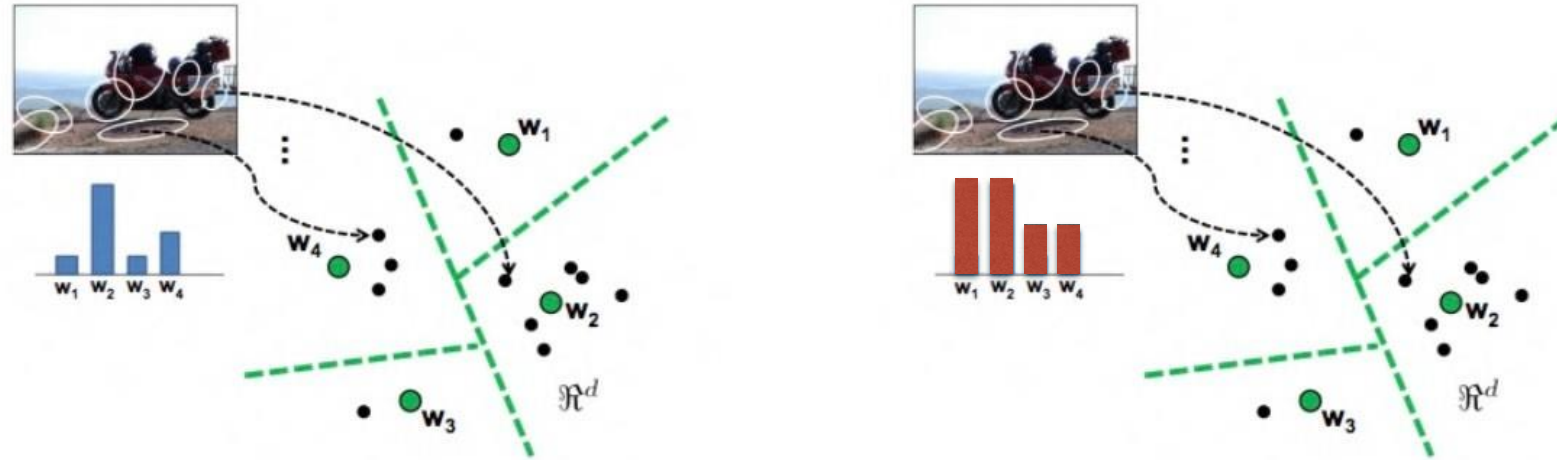
- 1) Extract features and descriptors in image
- 2) Discretize feature space (clustering)
- 3) Store the frequency of the features (visual words) for each image



Each cluster is a “visual word”

Typically 5k-10k (up to 100k) visual words

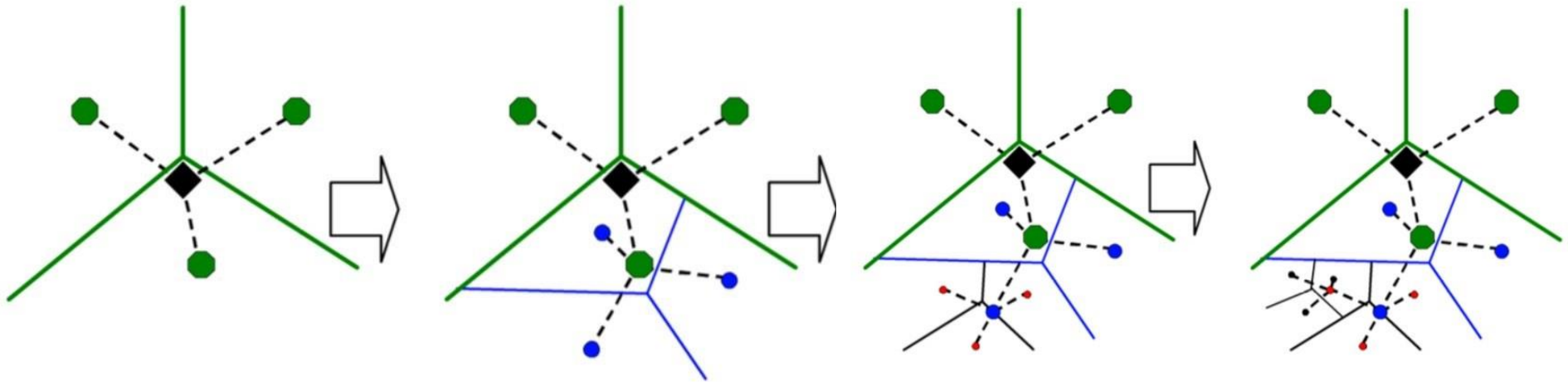
Local descriptors: Bag of Words



Two images are compared based on the corresponding histogram (Hamming distances, other metrics, ...)

Need Large Vocabularies: Vocabulary Tree

- ▶ Faster quantization (logarithmic time complexity in vocabulary size)
- ▶ Therefore can afford larger vocabularies
- ▶ Hierarchical clustering:



Credit: Nister and Stewenius

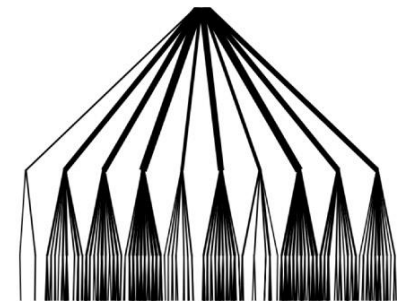
Scalable recognition with a vocabulary tree

D Nister, H Stewenius

2006 IEEE Computer Society Conference on Computer Vision and Pattern ...

4135

2006



What is VLAD ?

Vector of Locally Aggregated Descriptors

- BoW: count the number of descriptors associated with each cluster in a vocabulary (word set) and creating a histogram for each set of descriptors from an image.
- **VLAD**: Extension of BoW
- We accumulate the residual of each descriptor with respect to its assigned cluster.

If x_i are local descriptors, and c_j are cluster centers:

$$v_j = \sum_{x_i \in C_j} (x_i - c_j)$$

Final descriptor:

$$\text{VLAD} = [v_1, v_2, \dots, v_K]$$

We match a descriptor to its closest cluster, then for each cluster, we store the sum of the differences of the descriptors assigned to the cluster and the centroid of the cluster.

BoW-based Loop-Closure Detection in Action

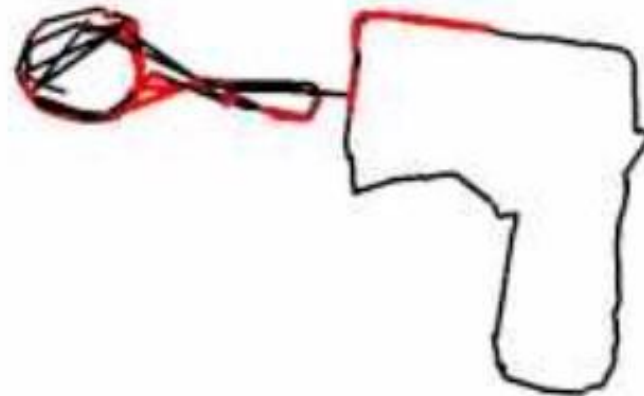
Current image



Loop detected



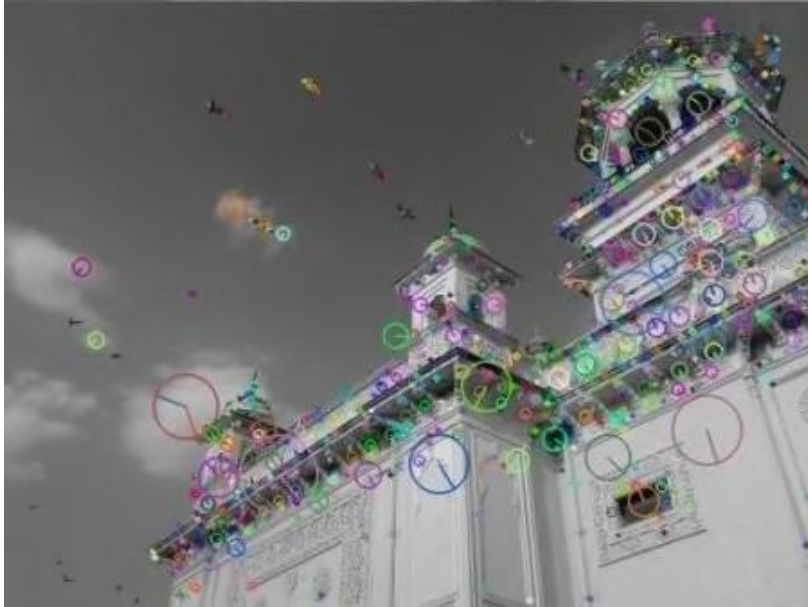
Execution time: 23.2 ms



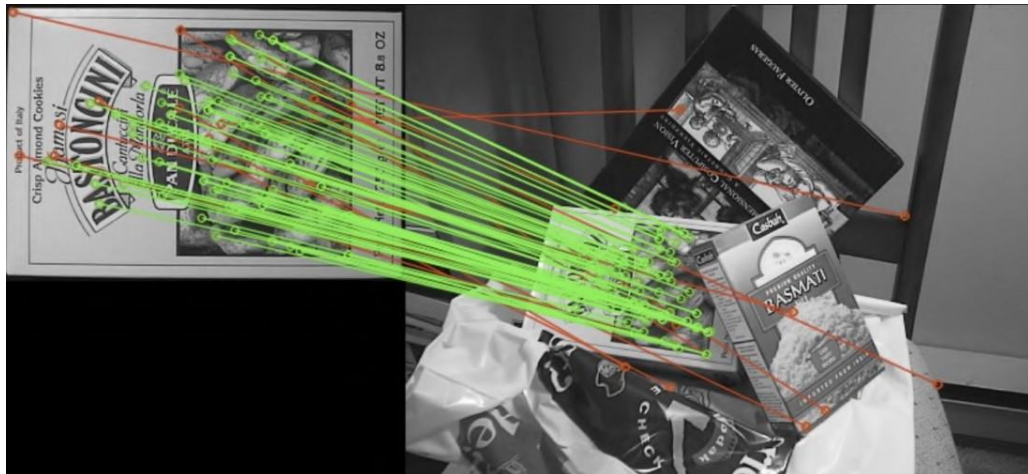
2012

Local descriptors

SIFT, SURF, ORB, Brief, ...



Naive approach:
stack all descriptors
in a vector

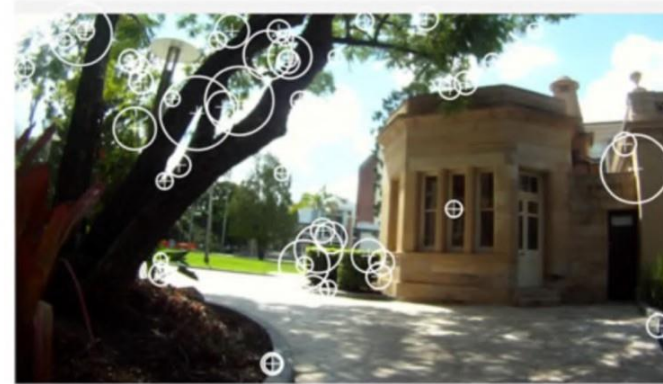


Is this a good
image descriptor?



Image Retrieval: Approaches

- Global descriptors
 - A feature representation that summarizes the *entire image* (or object) as a whole. It captures overall shape, color, or texture statistics.
- Local descriptors
 - A feature representation extracted from specific regions, points, or patches within the image. It captures fine-grained local details.
- Learning-based methods



(a)



(b)

[courtesy of Lowry'16]

Global descriptors

Early approaches:

- color histograms
- principal component analysis
- other statistics on edges, corners, and color patches

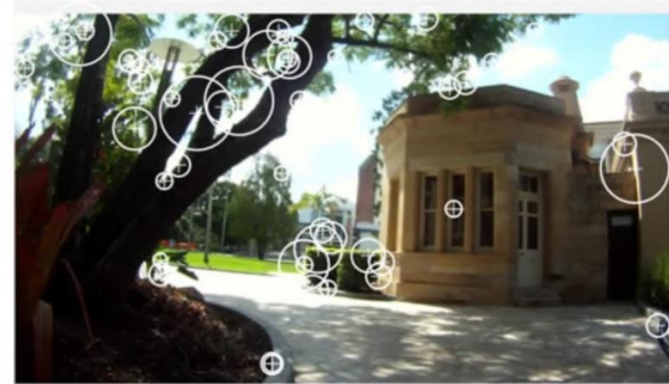


Early 2000:

- **GIST descriptor:**
 - image is filtered at different orientations and different frequencies to extract information from the image
 - results are averaged to generate a compact vector that represents the “gist” of a scene

Image Retrieval: Approaches

- Local descriptors
- Global descriptors
- Learning-based methods



(a)



(b)

[courtesy of Lowry'16]

Local vs. Global Descriptors

Local descriptors:

- allow estimating feature (and camera) geometry
- sensitive to lighting conditions and seasonal variations

Global descriptors:

- better at handling lighting conditions and seasonal variation
- more sensitive to viewpoint changes

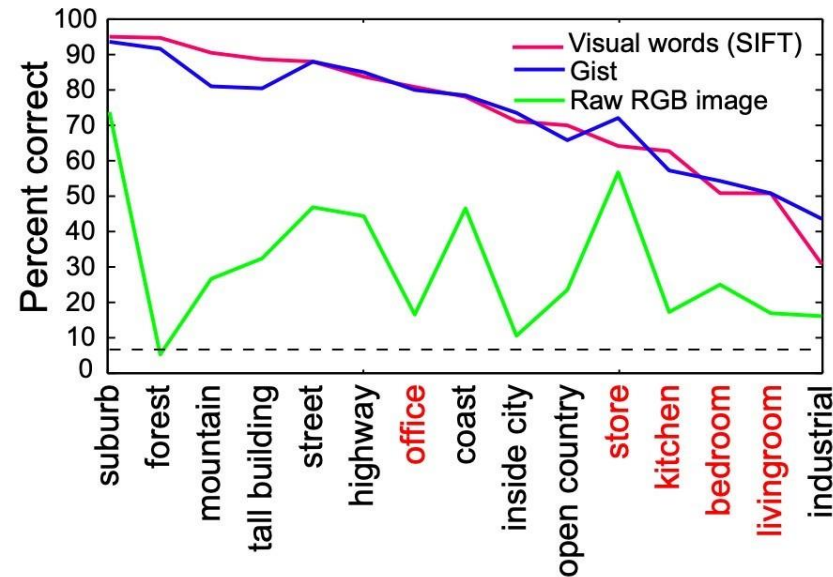


Figure 1. Comparison of Spatial Sift and Gist features for a scene recognition task. Both set of features have a strong correlation in the performance across the 15 scene categories. Average performance for the different features are: Gist: 73.0%, Pyramid matching: 73.4%, bag of words: 64.1%, and color pixels (SSD): 30.6%. In all cases we use an SVM.

Deep Learning Revolution

- new take on algorithms
- large amount of data
- large amount of computing (GPU)

2010: ImageNet Large Scale Visual Recognition Challenge (ILSVRC) is launched



14M images

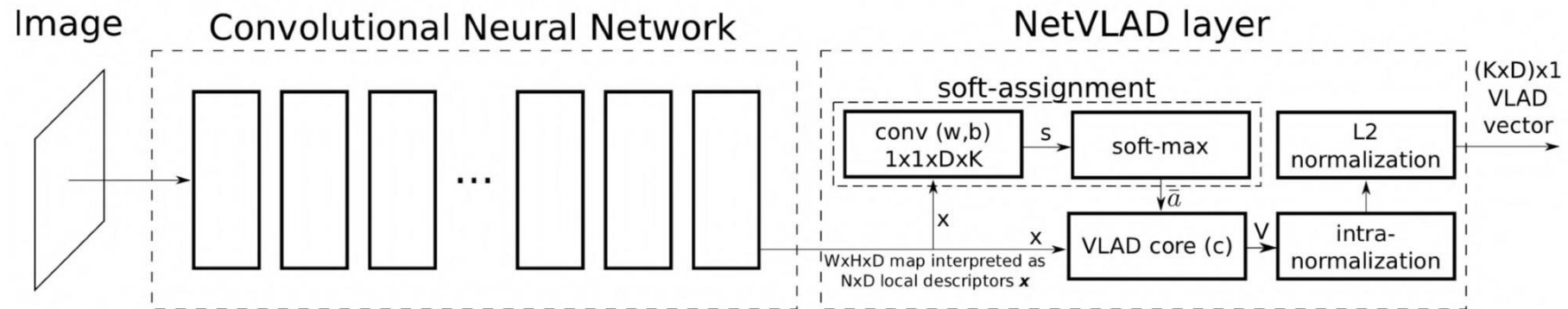
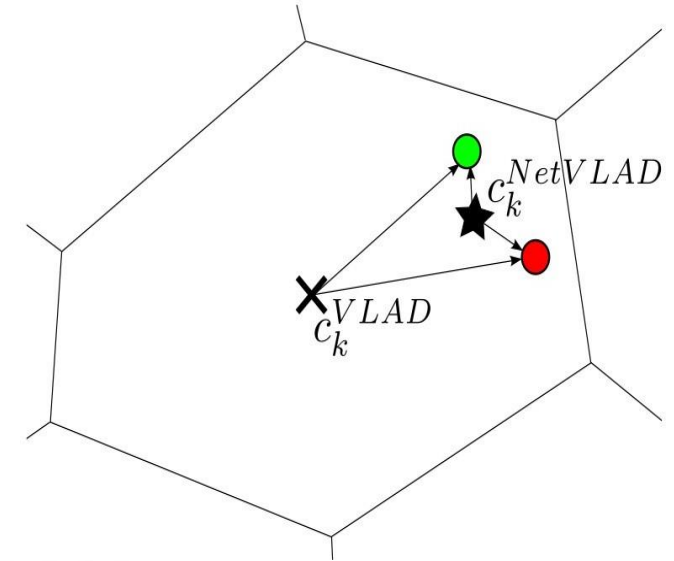
objects/
bounding
boxes

>1k classes

Learning-based Descriptors: NetVLAD

NetVLAD:

- CNN-based approach
- Trained on the task of place recognition
- Clever use of Internet data for training



Learning-based Descriptors: NetVLAD

How to get labeled data?

- a large dataset of panoramic images from the Google Street View Time Machine
- positions based on their (noisy) GPS
- Seasonal variations
- Illumination changes

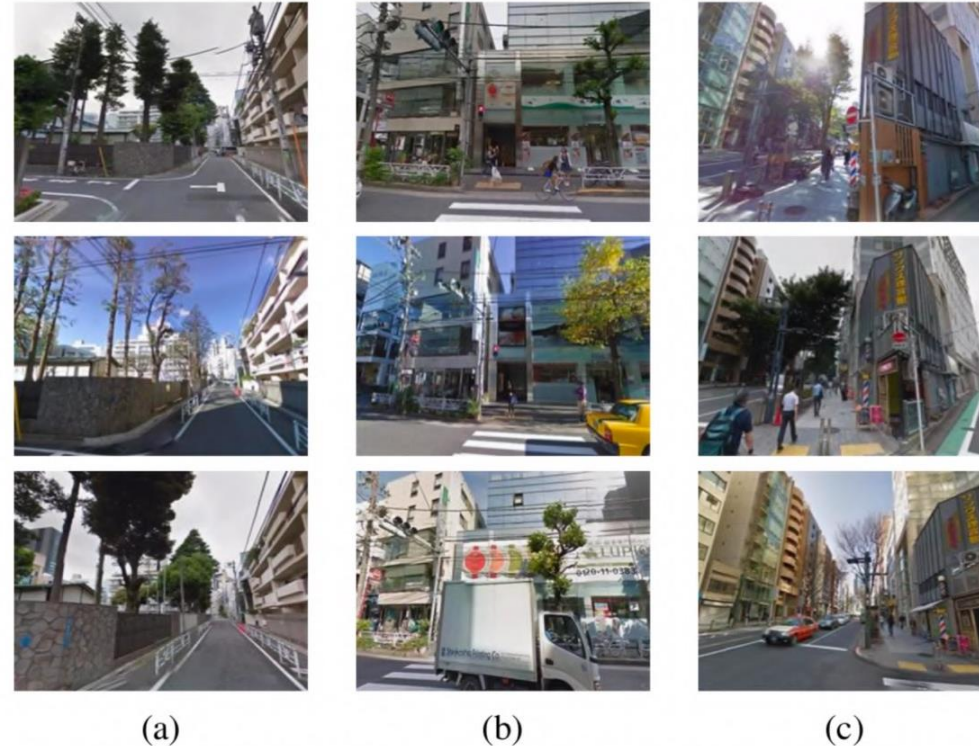
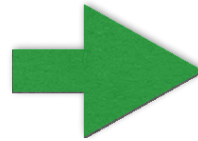
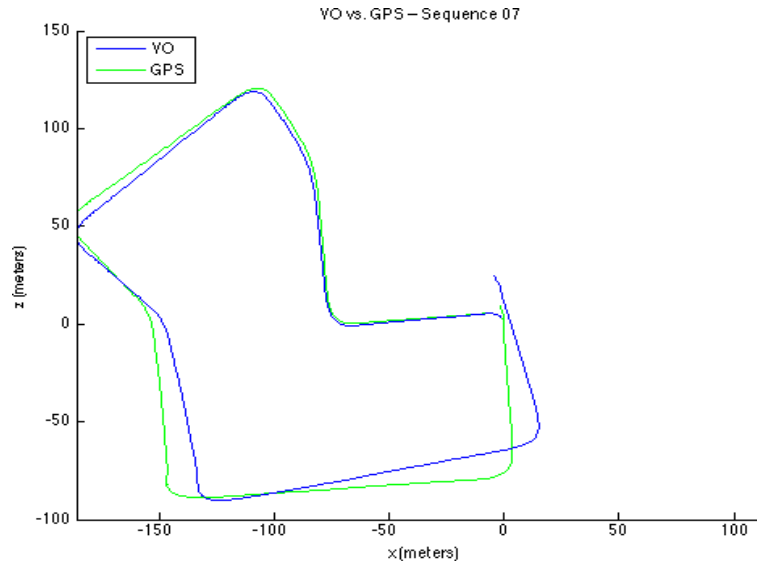


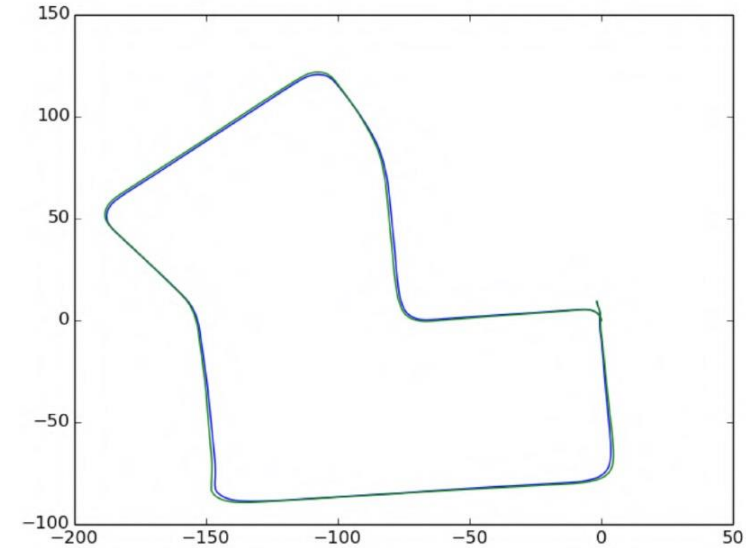
Figure 4. **Google Street View Time Machine examples.** Each column shows perspective images generated from panoramas from nearby locations, taken at different times. A well designed method can use this source of imagery to learn to be invariant to changes in viewpoint and lighting (a-c), and to moderate occlusions (b). It can also learn to suppress confusing visual information such as clouds (a), vehicles and people (b-c), and to chose to either ignore vegetation or to learn a season-invariant vegetation representation (a-c). More examples are given in appendix [B](#).

Removing Drift via Loop Closure

Visual odometry



SLAM



SLAM (Simultaneous Localization and Mapping) requires:

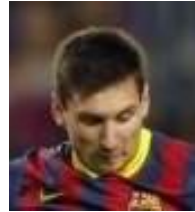
- place recognition => loop closure detection
- and / or

- Object detection => landmark detection

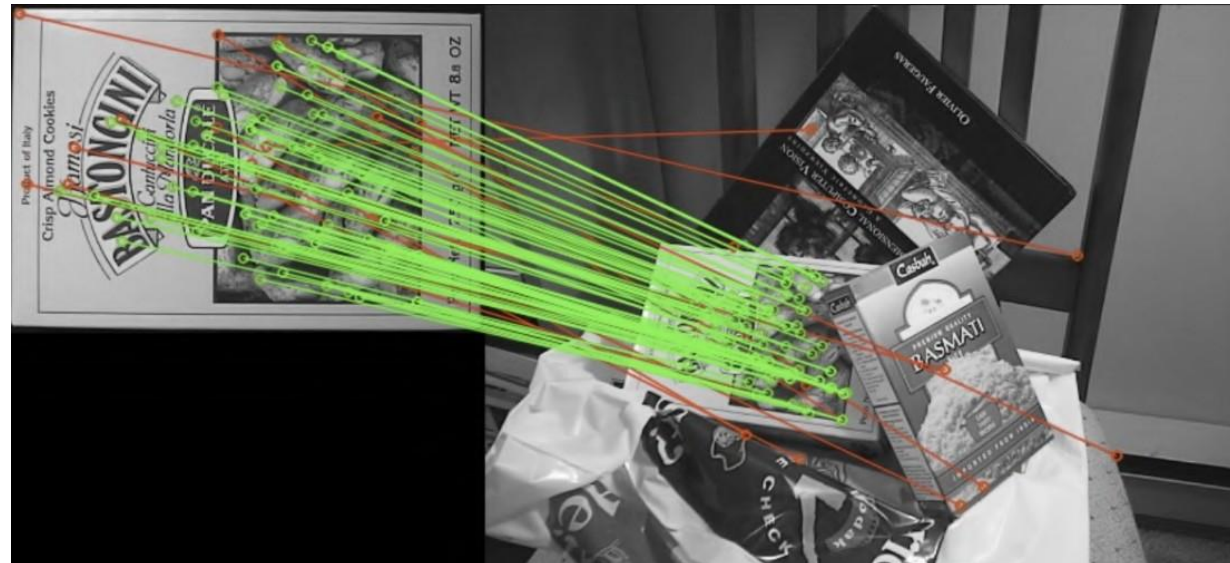
Traditional Object Detectors

- template matching
(sliding window)

template



- feature-based



(scalability?)

YOLO



Redmond et al, "You Only Look Once: Unified, Real-Time Object Detection", CVPR'16.

<https://www.youtube.com/watch?v=uG2UOaslx2I>

Learning-based Object Detection: YOLO

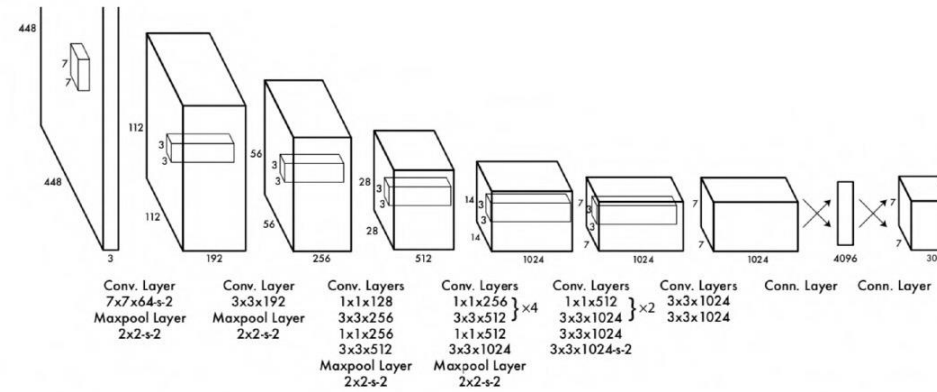


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

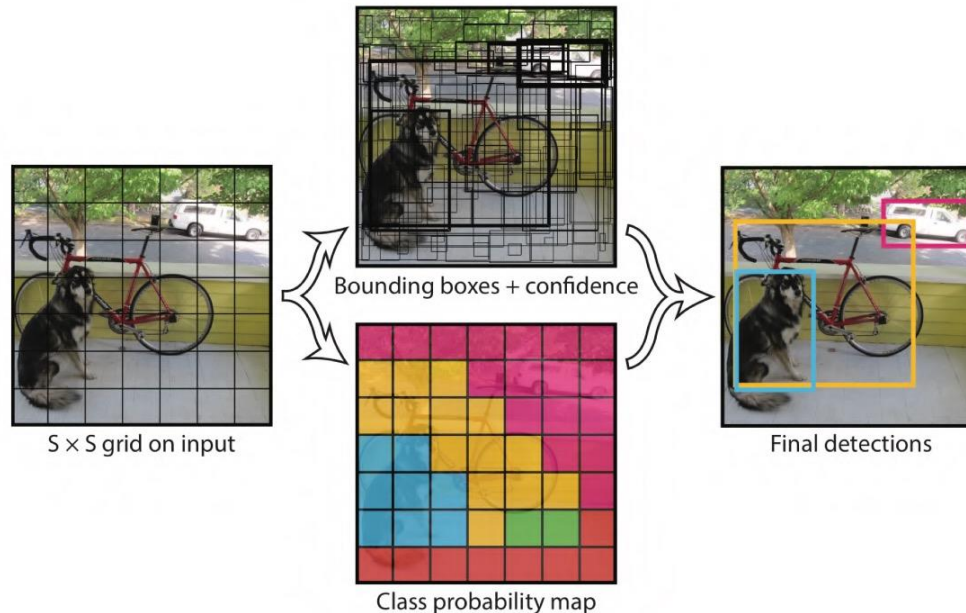


Image is split in $S \times S$ grid.

Yolo is trained to predict:

- B bounding boxes in each grid cell (x, y, h, w, confidence)
- A class label for each cell

Learning-based Object Detection: YOLO

mAP:

mean Average Precision (average precision value for recall value over 0 to 1).

- Limitations of YOLO:

- small objects: “each grid cell only predicts B boxes and can only have one class. This spatial constraint limits the number of nearby objects that our model can predict. Our model struggles with small objects that appear in groups, such as flocks of birds.”

- generalization: fails to detect objects in new or unusual aspect ratios or configurations.

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Table 1: Real-Time Systems on PASCAL VOC 2007. Comparing the performance and speed of fast detectors. Fast YOLO is the fastest detector on record for PASCAL VOC detection and is still twice as accurate as any other real-time detector. YOLO is 10 mAP more accurate than the fast version while still well above real-time in speed.

SLAM and Factor Graph Formulation

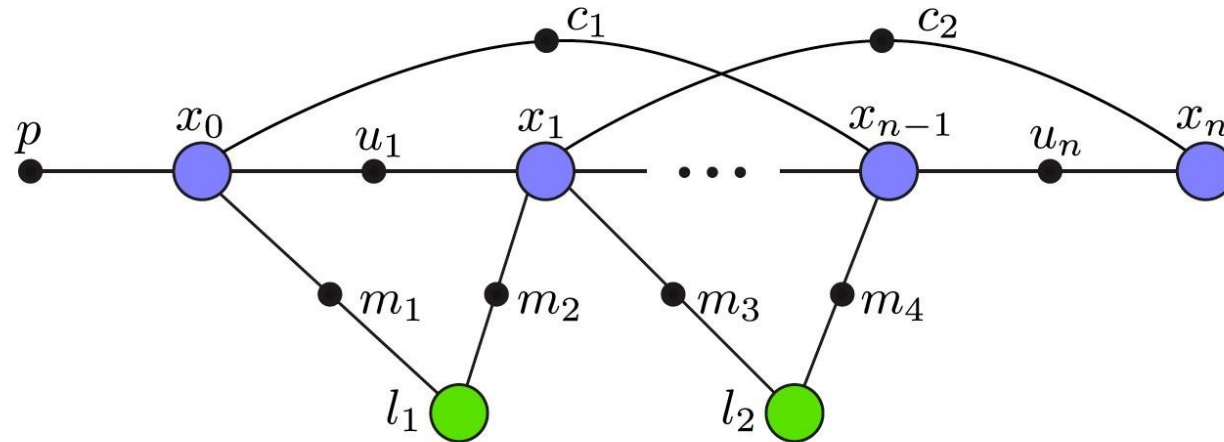


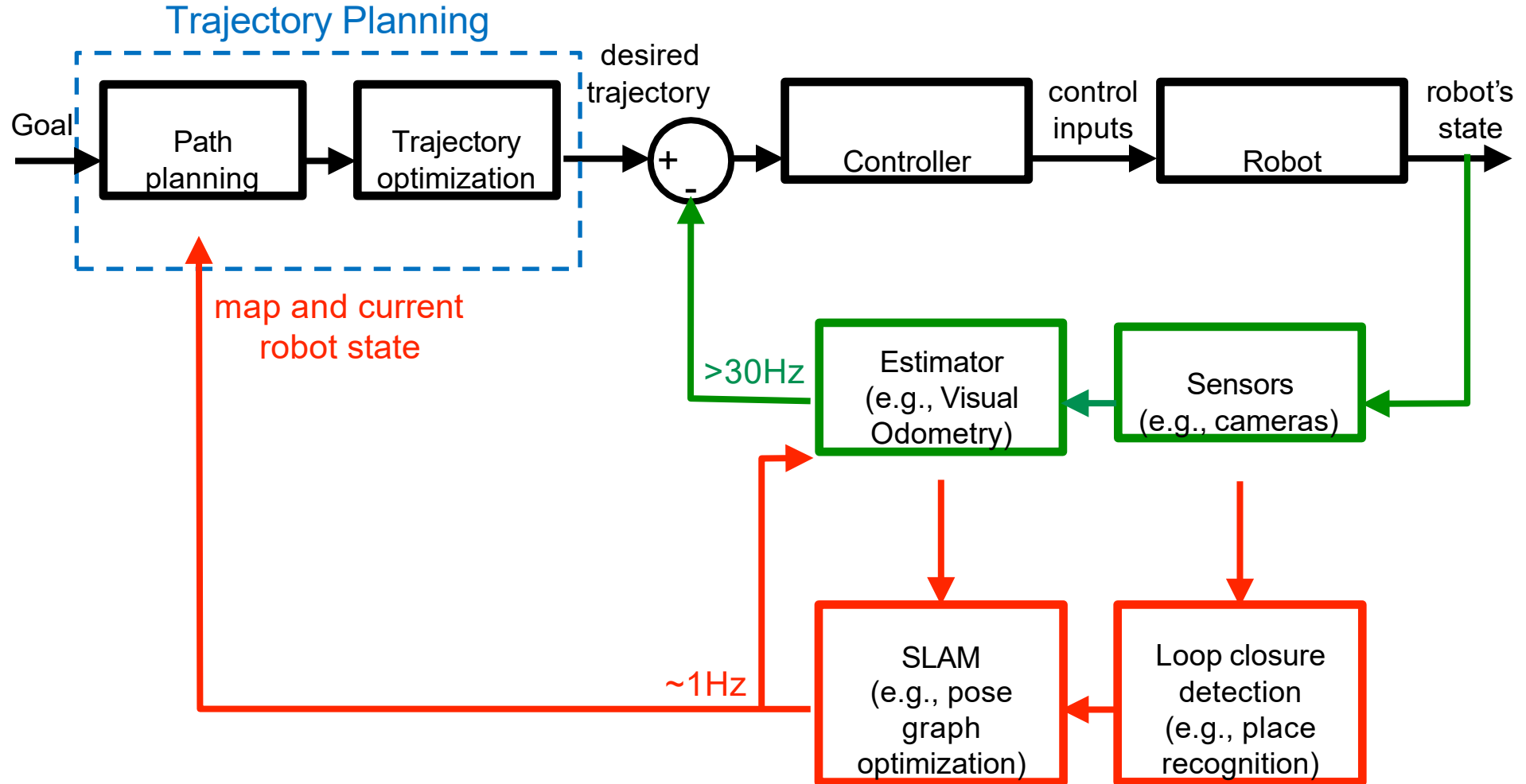
Fig. 2: Factor graph (Kschischang et al., 2001) formulation of the SLAM problem, where variable nodes are shown as large circles, and factor nodes (measurements) as small solid circles. The factors shown are odometry measurements u , a prior p , loop closing constraints c and landmark measurements m . Special cases include the pose-graph formulation (without l and m) and landmark-based SLAM (without c). Note that the factor graph can represent any cost function, involving one, two or more variables (e.g. calibration).

iSAM: Incremental Smoothing and Mapping

(iSAM), which performs fast incremental updates of the square root information matrix, yet is able to compute the full map and trajectory at any time.

iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree

Big Picture

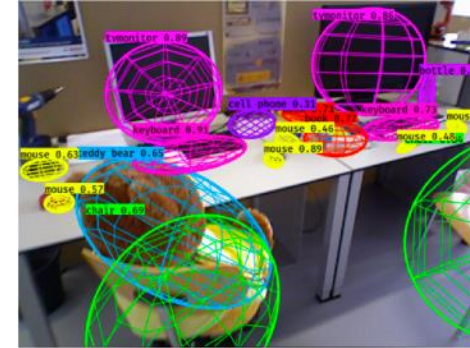


“Map”: Environment Representations

- **Sparse:**
 - Landmark-based
 - No explicit representation (pose graph)
 - Geometric primitives
- **Dense:**
 - Point clouds
 - 2D/3D occupancy grids
 - 3D meshes
 -



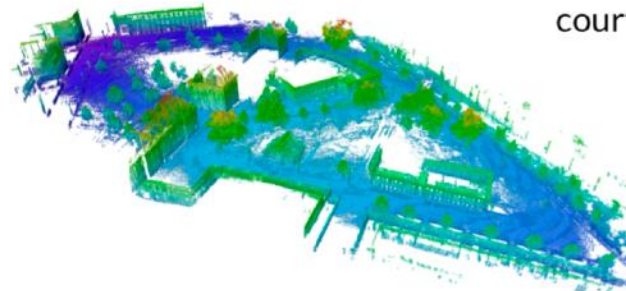
courtesy of Ranganathan et al.



courtesy of Nicholson et al.



courtesy of Sameer Agarwal

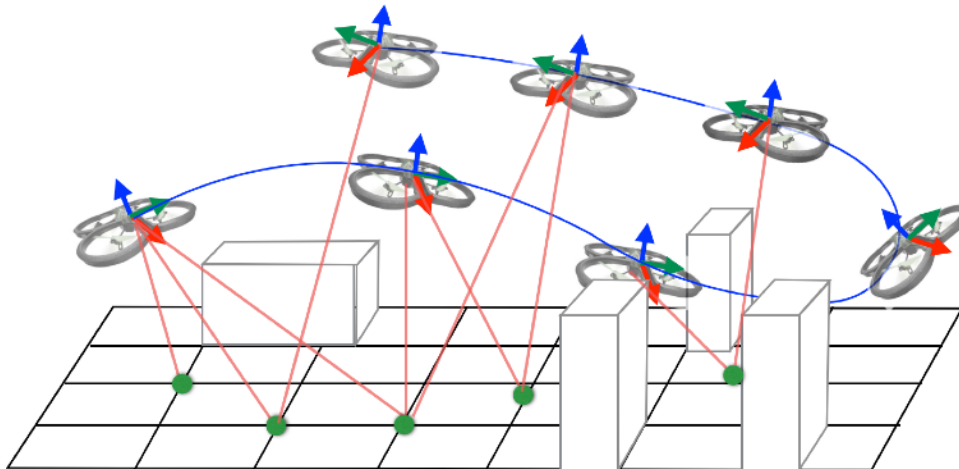
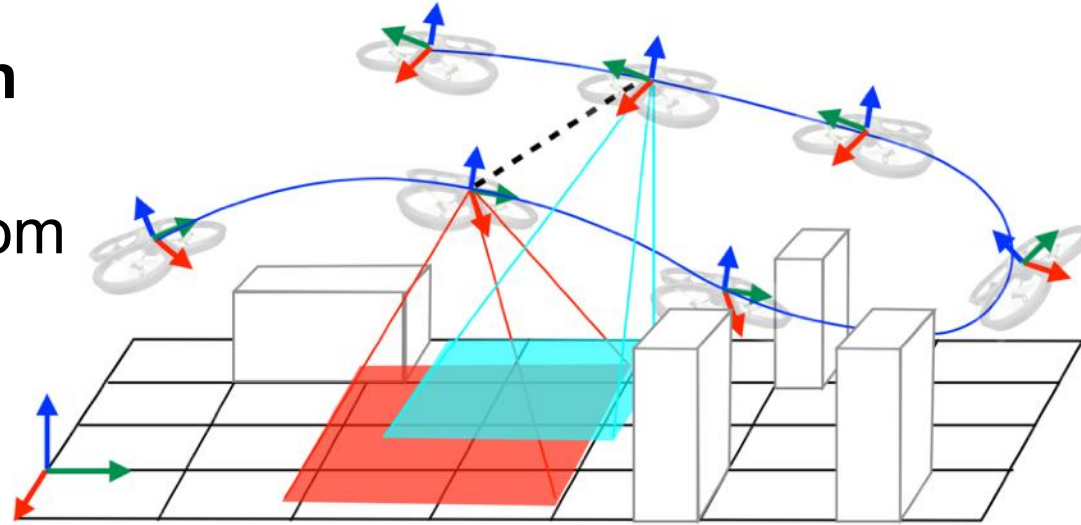


courtesy of octomap

Simultaneous Localization and Mapping

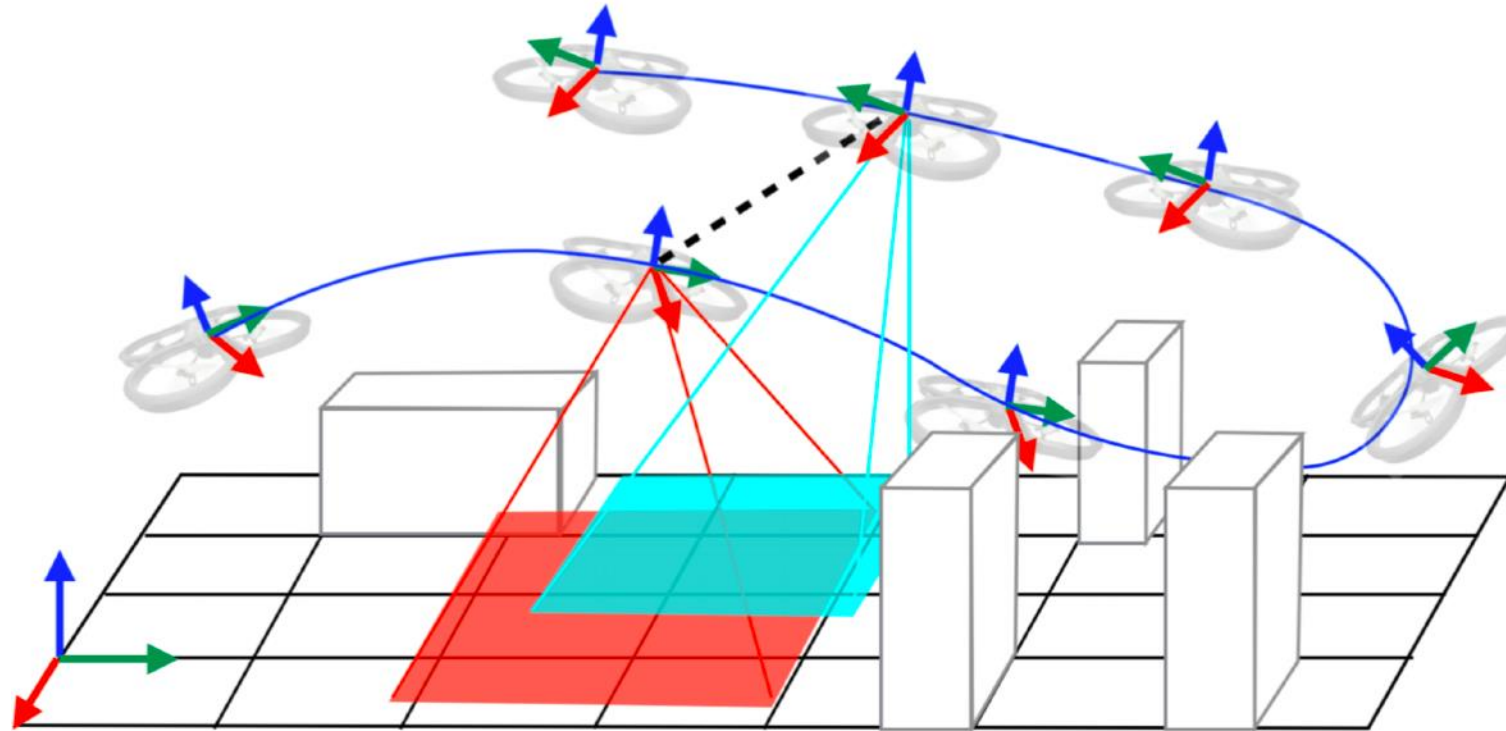
Pose Graph Optimization

(a.k.a. pose SLAM):
Estimate only trajectory from
sensor data



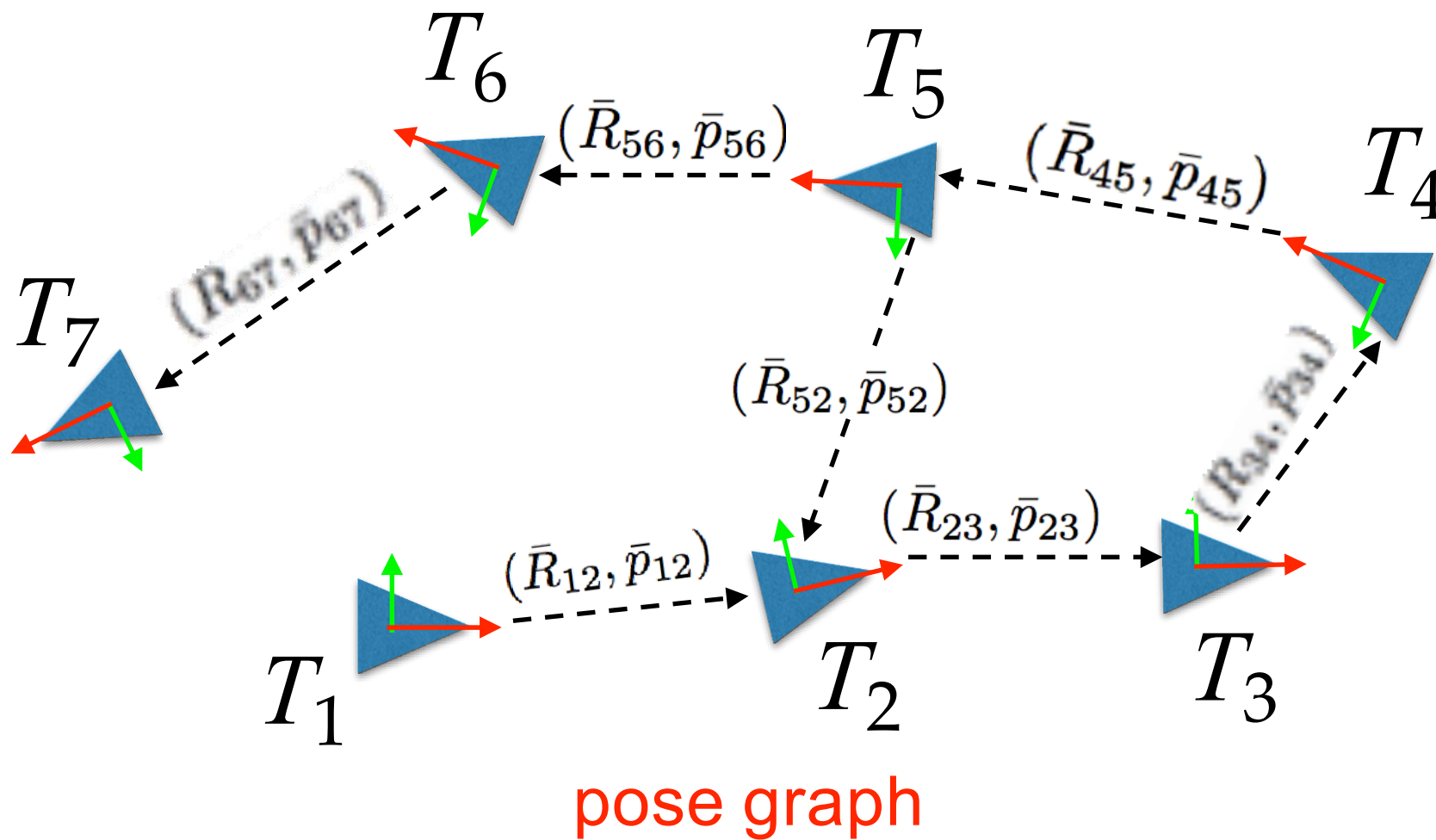
Landmark-based SLAM:
Estimate trajectory of robot
and position of external
landmarks from sensor
data

Pose Graph Optimization

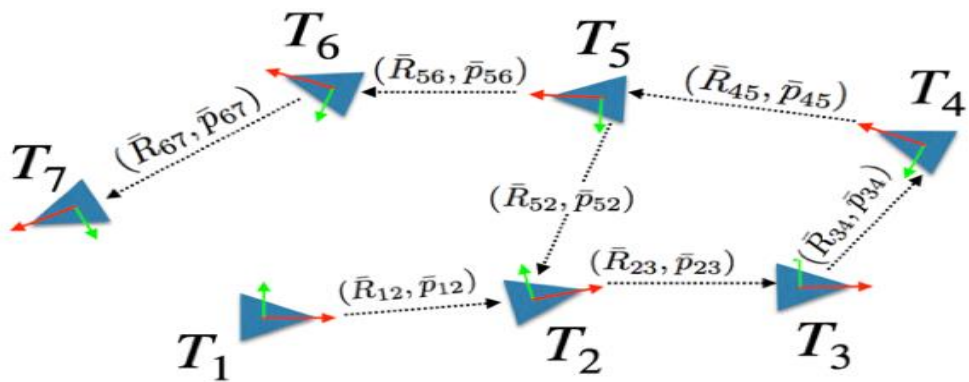


- **Measurements:** odometry + loop closures (i.e., relative pose measurements between non-consecutive poses obtained via place recognition & 2-view geometry, or similar)
- **Variables:** robot poses

Graphical representation of pose graph optimization



Pose Graph Optimization: Sparsity



Jacobian $\mathbf{J}$

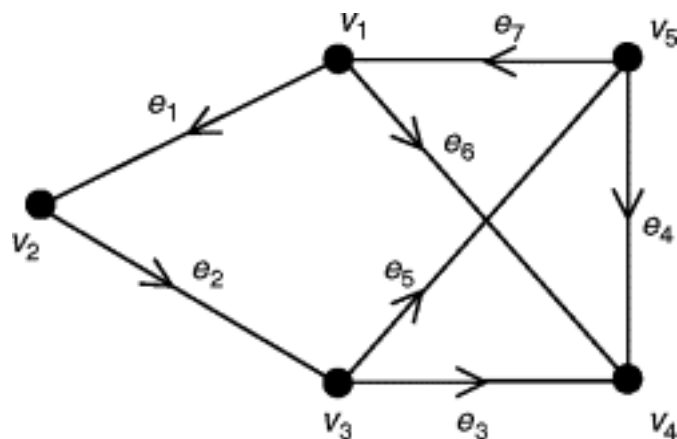
	T_1	T_2	T_3	T_4	T_5	T_6	T_7
$\bar{T}_{12}$	■	■					
$\bar{T}_{23}$		■	■				
$\bar{T}_{34}$			■	■			
$\bar{T}_{45}$				■	■		
$\bar{T}_{56}$					■	■	
$\bar{T}_{67}$						■	■
$\bar{T}_{52}$		■			■		

Hessian $\mathbf{J}^T \mathbf{J}$

	T_1	T_2	T_3	T_4	T_5	T_6	T_7
T_1	■	■					
T_2	■	■	■		■		
T_3		■	■	■			
T_4			■	■	■		
T_5		■		■	■	■	
T_6					■	■	■
T_7						■	■

a.k.a.
Information
Matrix of
the estimate

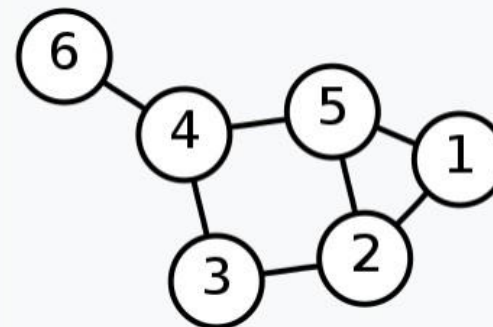
Graphical representation of pose graph optimization



Incidence Matrix

	e_1	e_2	e_3	e_4	e_5	e_6	e_7
v_1	1	0	0	0	0	1	-1
v_2	-1	1	0	0	0	0	0
v_3	0	-1	1	0	1	0	0
v_4	0	0	-1	-1	0	-1	0
v_5	0	0	0	1	-1	0	1

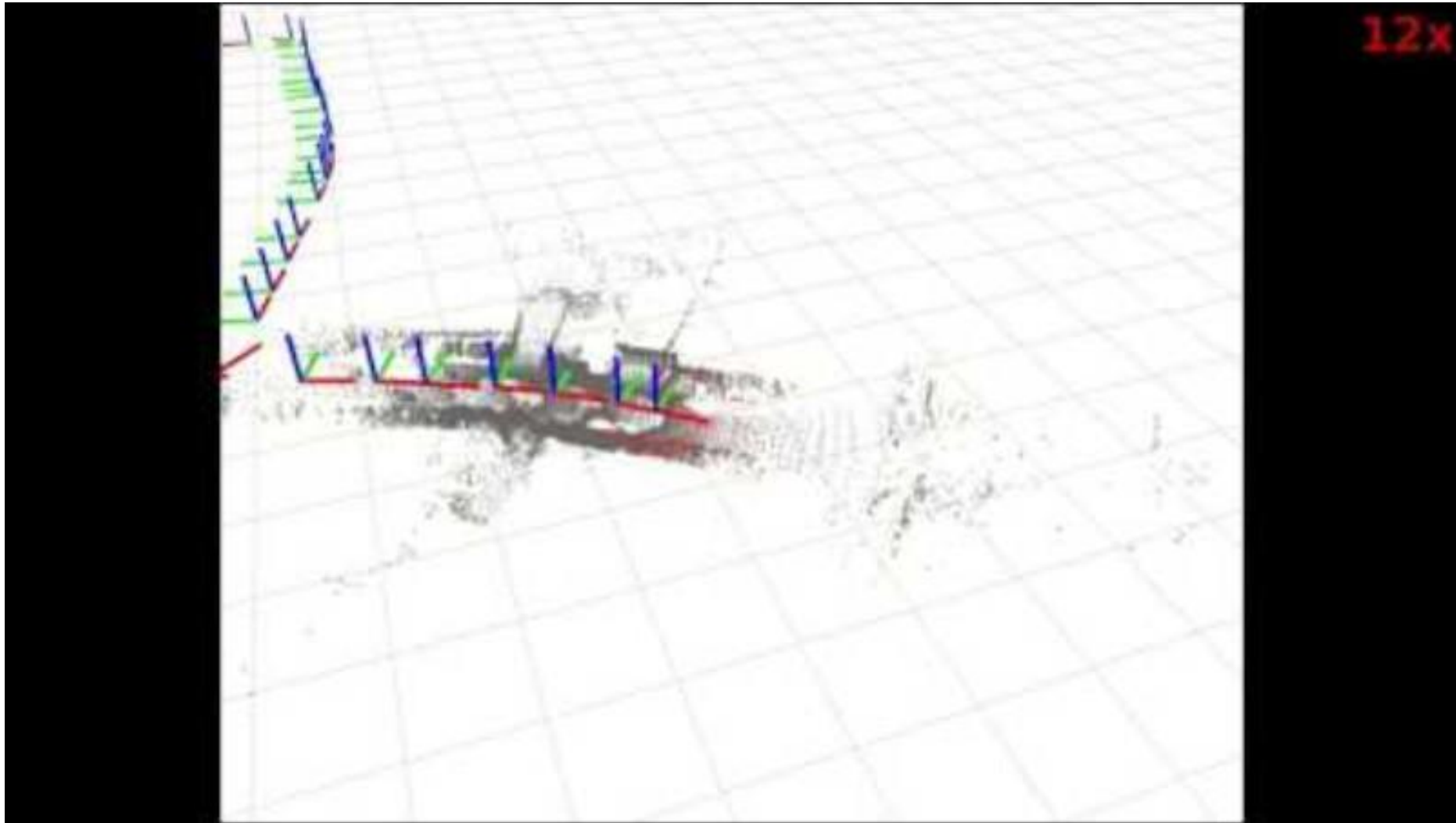
Labelled graph



Laplacian matrix

$\begin{pmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{pmatrix}$
--

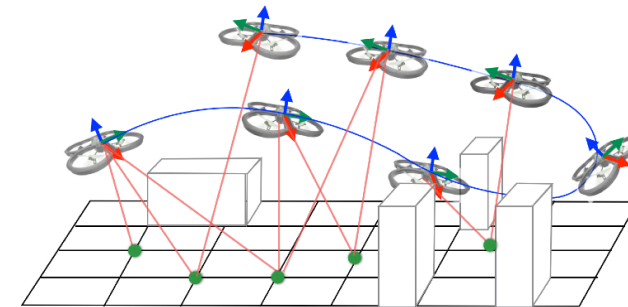
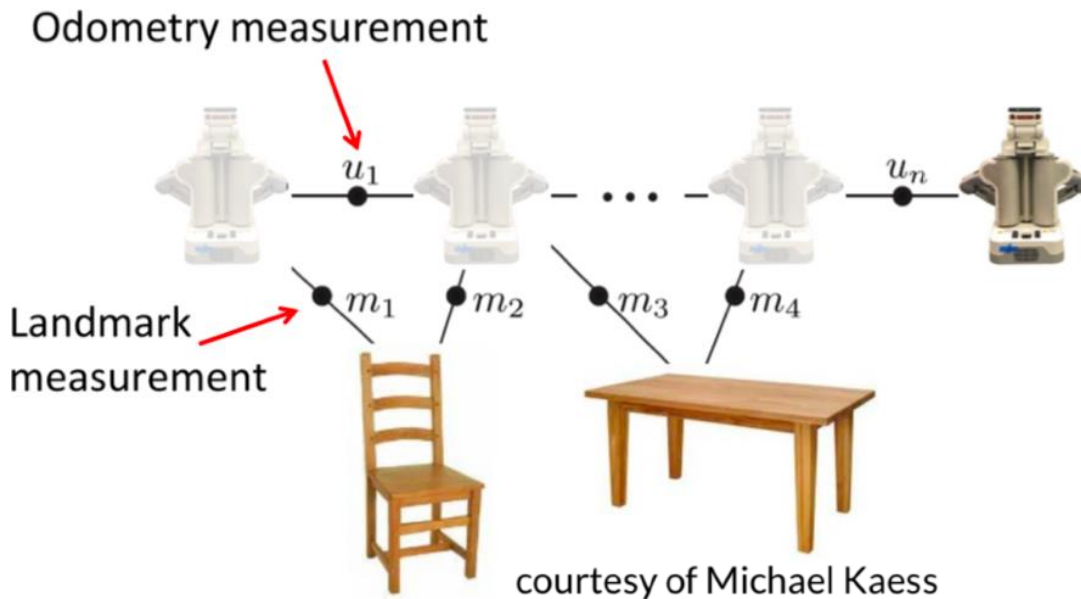
Pose Graph Optimization: Example



https://www.youtube.com/watch?v=KYvOqUB_odg

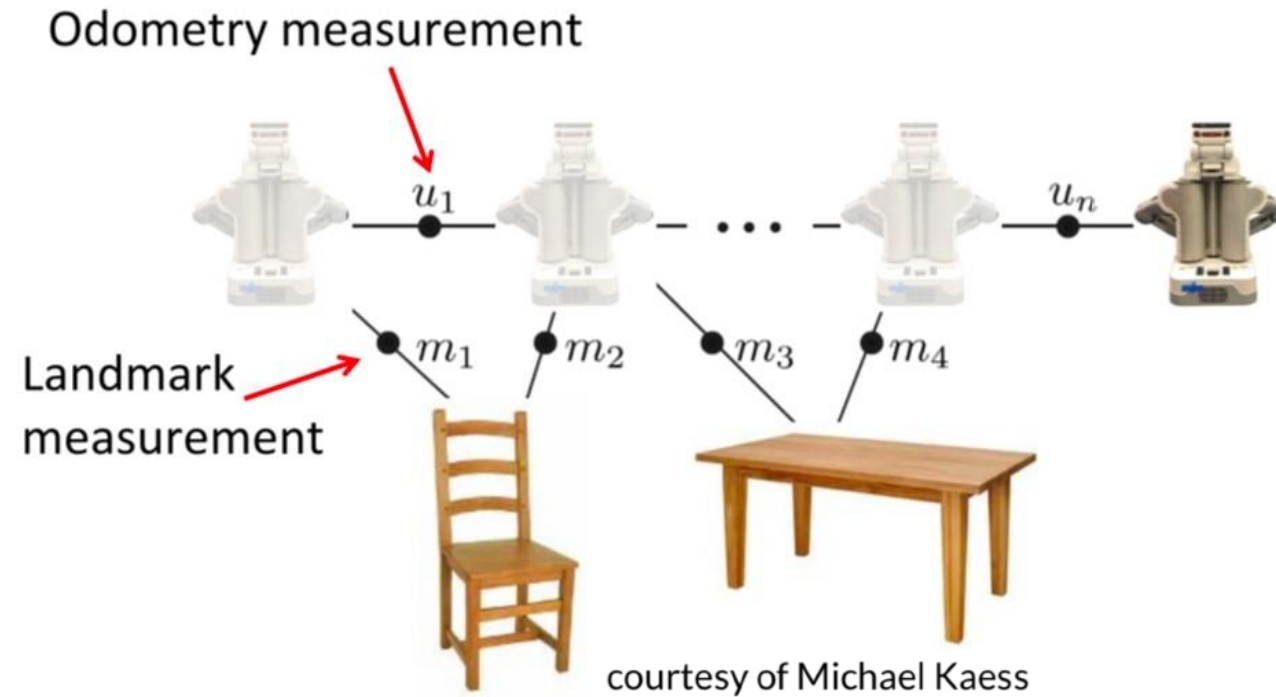
Landmark-based SLAM

- ▶ Sequence of robot (camera) poses $\mathbf{T}_1, \mathbf{T}_2, \dots, \mathbf{T}_t \in \text{SE}(d)$
- ▶ Robot measures the relative pose between $\mathbf{T}_i$ and $\mathbf{T}_{i+1}$ (odometry)
- ▶ Robot measures the environment (e.g., point landmarks $\mathbf{p}_i \in \mathbb{R}^d$)



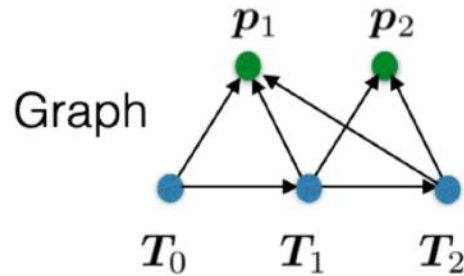
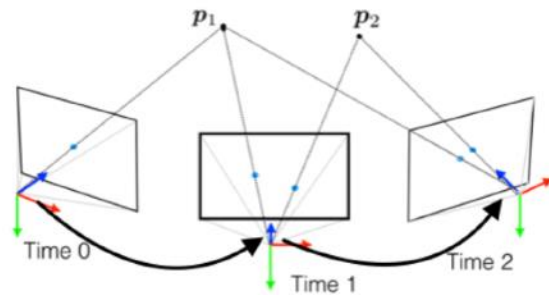
- **Measurements:** odometry + measurements of (projection, range, position, or others) of external landmarks
- **Variables:** robot poses and landmark positions

Graphical representation of landmark-based SLAM



- ▶ Each variable (robot pose, landmark position/pose) is a node in the graph
- ▶ Each (usually) pairwise measurement denotes an edge between the corresponding two variables (nodes)

Graphical representation of landmark-based SLAM



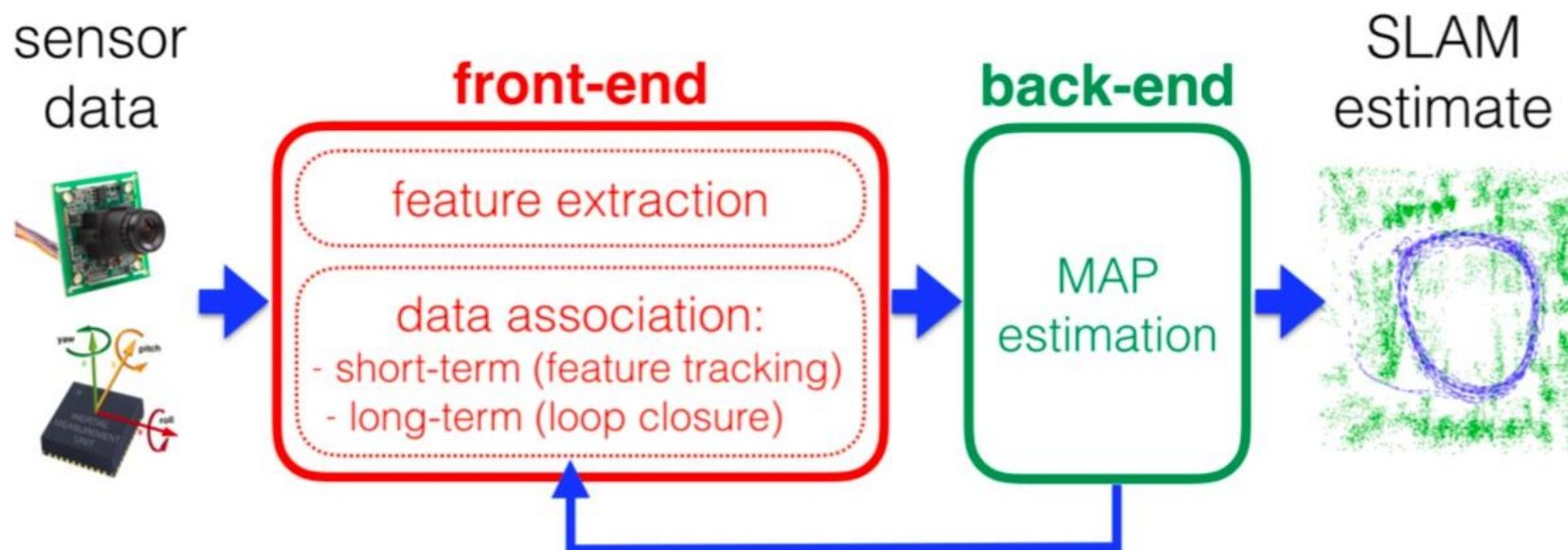
Jacobian $\mathbf{J}$

	T_0	T_1	T_2	p_1	p_2
T_0	Blue	Blue	White	White	White
T_1	White	Blue	Blue	White	White
T_2	Blue	White	Blue	Blue	White
p_1	White	Blue	White	Blue	White
p_2	White	White	Blue	Blue	Blue

Hessian $\mathbf{J}^T \mathbf{J}$

	T_0	T_1	T_2	p_1	p_2
T_0	Dark Blue	Dark Blue	White	Light Blue	White
T_1	Dark Blue	Dark Blue	Dark Blue	Light Blue	Light Blue
T_2	White	Dark Blue	Dark Blue	Light Blue	Light Blue
p_1	Light Blue	Light Blue	Light Blue	Dark Blue	White
p_2	White	Light Blue	Light Blue	White	Dark Blue

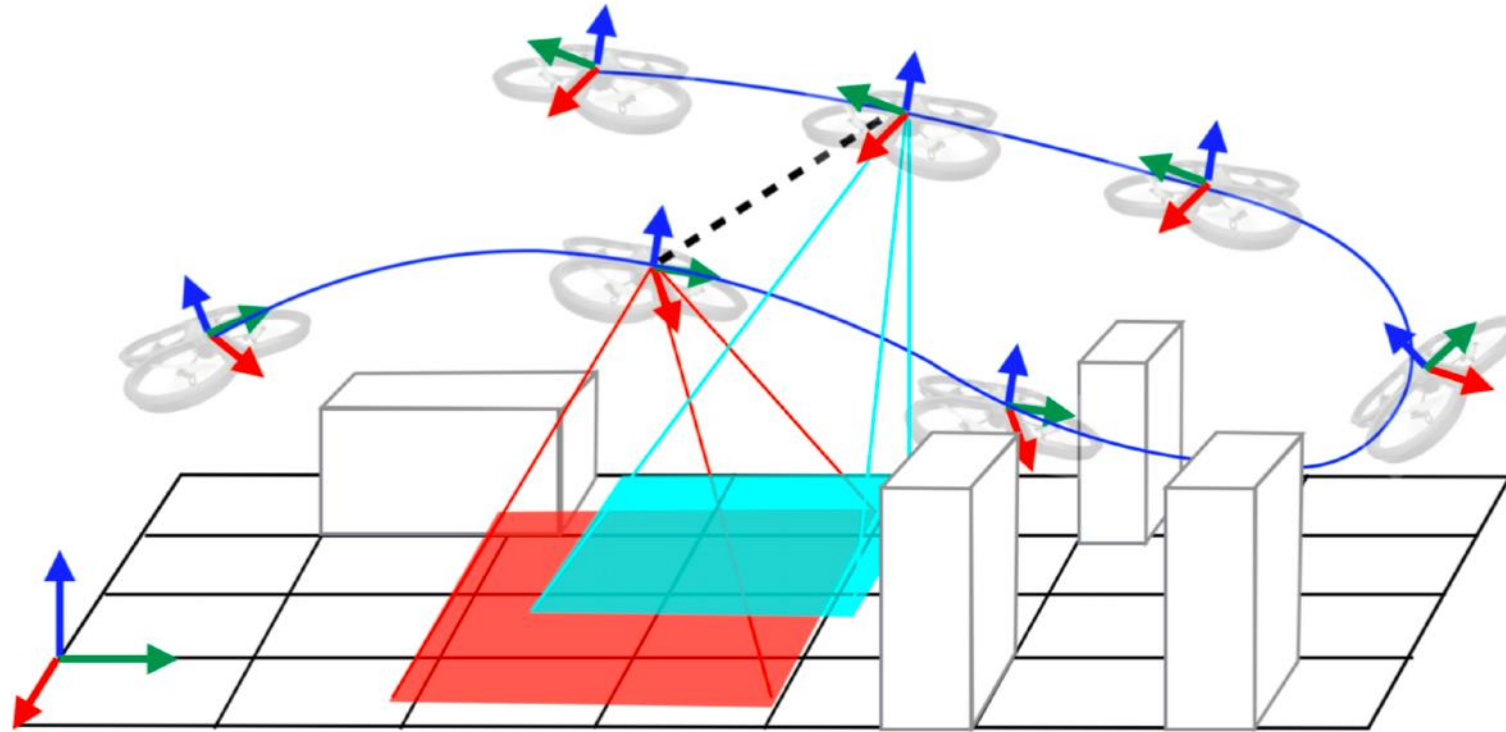
Some terminology



MAP is maximum *a posteriori* estimation
(MLE if no prior is available [“uninformative” prior])

courtesy of Cadena et al.

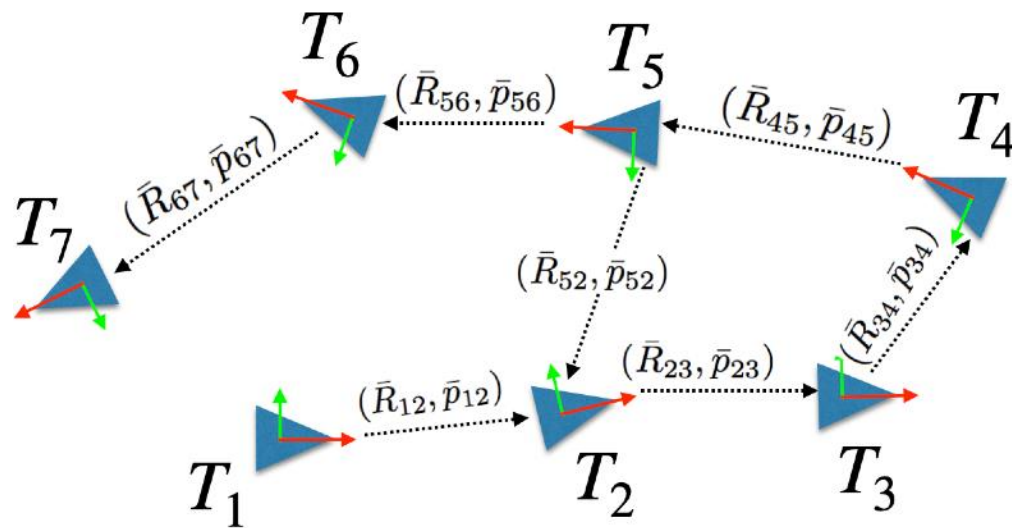
Pose Graph Optimization Recap



- **Measurements:** odometry + loop closures (relative poses)
- **Variables:** robot poses

Pose Graph Optimization

pose
graph



pose graph
optimization

$$\min_{\mathbf{T}_t, t=1, \dots, n} \sum_{(i,j) \in \mathcal{E}} \|(\mathbf{T}_i^{-1} \mathbf{T}_j) \ominus \bar{\mathbf{T}}_j^i\|_{\Sigma_{ij}}^2$$